



Comunidad de Madrid



ARQUITECTO
VENTURAN
RODRÍGUEZ

La falta de aprendizaje real en los LLMs

Autor: Bruno Visdómine Minguito

Tutor: Jaime Martín García-Cueva

IES Arquitecto Ventura Rodríguez

Curso 2025/2026

BACHILLERATO
DE EXCELENCIA **B.**



Comunidad de Madrid

La falta de aprendizaje real en los LLMs

La historia de la Inteligencia Artificial, las diferencias con la inteligencia humana y la demostración de la falta de aprendizaje real y continuo en los LLMs.

Este proyecto ha sido realizado en el programa de Bachillerato de Excelencia del IES
Arquitecto Ventura Rodríguez

Año 2025

[Creative commons aqui](#)



Agradecimientos

Quisiera dar las gracias a todas las personas que me han ayudado a llevar a cabo este proyecto, es probable que no me sea posible nombrar a cada persona implicada, pero me gustaría agradecer a todo aquel que haya aportado algo, su ayuda o motivación ya que cada pequeño empujón o incluso muestra de curiosidad pueden hacer una gran diferencia.

Principalmente me gustaría agradecer a mi madre y a mi padre el interés, la ayuda inconmensurable y los recursos que me han dado, haciendo mi curiosidad e interés en el proyecto cada vez más grande, simplemente hay que mirar a la gran cantidad de información e ideas han tenido que ser quitadas del artículo final debido a la falta de espacio. En segundo lugar, me gustaría agradecer el interés y la ayuda proporcionada por mi tutor Jaime Martín García-Cueva, quién ha seguido en contacto conmigo incluso mientras me encontraba en otro país, también le debo bastante a mi profesora Yolanda Aguilar, ya que me ha orientado y ayudado durante gran parte del proyecto manteniéndome en contacto con mi tutor y recordándome los plazos.

Me gustaría, en tercer lugar, agradecer al centro el esfuerzo realizado para la correcta realización del trabajo y su apoyo, hay que añadir que sin el programa de excelencia nunca se me habría ocurrido la idea de hacer un trabajo de investigación. Aquí me gustaría incluir a todos los miembros del centro que se hayan visto implicados.

Finalmente tengo que remarcar que nada de esto habría sido posible sin la ayuda y la base teórica establecida por muchos antes que yo que me han introducido al mundo del ML, la programación y las matemáticas.

Resumen

Este trabajo examina las limitaciones estructurales de los LLMs que son modelos diseñados para operar con el lenguaje y generar o traducir texto entre otros, para alcanzar una inteligencia artificial general. Se hace una introducción al funcionamiento de los algoritmos con mayor impacto para establecer una base teórica para el lector.

Mediante un análisis filosófico fundamentado en la distinción entre deducción, inducción y la abducción y su relación con la IA, se investiga por qué los modelos actuales, basados exclusivamente en inferencia estadística inductiva, carecen de capacidades de razonamiento abductivo y aprendizaje continuo. Se analiza la arquitectura Transformer y se realiza experimentación empírica con técnicas de adaptación como LoRA para evaluar la plasticidad paramétrica. El estudio explora las consecuencias de la fijación de parámetros tras el entrenamiento, el fenómeno del olvido catastrófico que consiste en la falta de retención de “conocimiento”, y las implicaciones que podría tener un enfoque basado en el aprendizaje continuo como el que se encuentra en la neuroplasticidad en relación con la AGI y la abducción. Además, se experimenta con una métrica que evidencia las limitaciones actuales en la plasticidad y capacidad de adaptación de los modelos. Finalmente, se proponen direcciones de investigación prometedoras en relación con los puntos desarrollados y los datos obtenidos.

Abstract

This study examines the structural limitations of large language models (LLMs) in achieving artificial general intelligence (AGI). It begins with a theoretical overview of the most influential algorithms to provide a conceptual foundation for the reader. Through a philosophical analysis grounded in the distinction between deduction, induction, and abduction, it argues that current models—based primarily on inductive statistical inference—lack the mechanisms required for abductive reasoning and continuous learning. The Transformer architecture is analyzed, and empirical experimentation is conducted using adaptation techniques such as LoRA to evaluate parametric plasticity. The study explores issues such as parameter fixation after training, catastrophic forgetting, and knowledge retention, relating them to the notion of neuroplasticity. Finally, it outlines promising research directions aimed at integrating principles of plasticity and abduction into future architectures oriented toward AGI.



Índice

Índice.....	5
1. INTRODUCCIÓN.....	6
2. ESTADO DEL ARTE.....	7
3. OBJETIVOS E HIPÓTESIS.....	8
3.1 Hipótesis.....	8
3.2 Objetivos del Proyecto.....	8
3.3 Ámbito, Alcance y Límites.....	9
4. FUNDAMENTOS TEÓRICOS.....	9
4.1 La Inteligencia desde la filosofía.....	9
4.1.1 Deducción.....	10
4.1.2 Inducción.....	10
Restricción empírica de los modelos basados en datos.....	11
La Regularidad.....	13
4.1.3 Abducción.....	13
La Abducción como Motor del Pensamiento Real.....	14
Limitaciones de la IA Actual (Frente a la Abducción).....	14
4.2 Modelos de ML.....	15
Transformers.....	15
4.3 Otros modelos y tipos de machine learning.....	20
Perceptrón.....	20
Perceptrón multicapa.....	22
CNN.....	23
RNN.....	24
Modelos de difusión.....	24
4.4 Neuroplasticidad y aprendizaje continuo biológico.....	24
Resumen Teórico.....	25
6. METODOLOGÍA Y FUENTES.....	27
7. EXPERIMENTACIÓN Y ANÁLISIS DE RESULTADOS.....	27
8. CONCLUSIONES.....	28
8.1. Evaluación.....	28
8.2 Divulgación.....	29
8.3 Futuras líneas de investigación.....	29
9. REFERENCIAS CONSULTADAS (bibliografía, páginas Web, etc.).....	29
10. ANEXOS.....	32

1. INTRODUCCIÓN

La explosión mediática en torno a la inteligencia artificial, especialmente tras la popularización de modelos como Chat GPT, ha generado expectativas desproporcionadas sobre la inminente llegada de una inteligencia artificial general (IAG). Este fenómeno de *hype* tecnológico no solo distorsiona la percepción pública sobre las capacidades reales de estos sistemas, sino que amenaza con desviar recursos y esfuerzos investigativos hacia soluciones comercialmente rentables a corto plazo, en detrimento de investigaciones fundamentales que podrían conducir a avances genuinos.

La motivación central de este trabajo surge de la necesidad de establecer una evaluación crítica y rigurosa de los límites estructurales de los grandes modelos de lenguaje (LLMs) actuales. Frente al discurso que presenta estos sistemas como pasos inevitables hacia la IAG, resulta imperativo examinar si la arquitectura predominante puede realmente conducir a una inteligencia comparable a la humana, o si representa un callejón sin salida técnico que requiere replanteamientos fundamentales. Comprender las limitaciones estructurales de los LLMs no solo tiene interés técnico, sino también filosófico y ético, pues redefine qué entendemos por aprender.

2. ESTADO DEL ARTE

Contexto Histórico: De la Lógica Simbólica al Aprendizaje Estadístico

El campo de la inteligencia artificial ha atravesado múltiples paradigmas desde su fundación formal en la conferencia de Dartmouth en 1956. Los primeros enfoques, inspirados en el trabajo de McCulloch y Pitts (1943) y posteriormente desarrollados por pioneros como Rosenblatt con el perceptrón, intentaron replicar la inteligencia aunque pronto encontraron limitaciones.

El llamado "invierno de la IA" de las décadas de 1970 y 1980 marcó el reconocimiento de estas limitaciones. Sin embargo, la disponibilidad creciente de datos masivos (*big data*) y capacidad computacional, permitió el entrenamiento de los modelos y la creación de nuevos algoritmos.

El Auge de los Transformers y los LLMs

La arquitectura Transformer, presentada por Vaswani et al. en 2017 en el influyente artículo "Attention is All You Need", revolucionó el procesamiento del lenguaje natural. Superando las limitaciones de las redes neuronales recurrentes (RNN) y las redes convolucionales en tareas secuenciales.

Esta arquitectura ha dado lugar a modelos cada vez más masivos como GPT-5 con 1,5 billones de parámetros. El éxito comercial de estos sistemas en tareas de generación de texto, traducción, resumen y conversación ha alimentado la narrativa de que la IAG es simplemente cuestión de escalar estos modelos con más datos y más parámetros.

Punto de Partida

El debate académico se polariza entre quienes consideran que la IAG emergerá simplemente de suficiente escala (Open AI, Anthropic), y quienes argumentan—como este trabajo—que se requieren innovaciones arquitectónicas fundamentales (DeepMind, IBM). La investigación se propone demostrar empíricamente estas limitaciones estructurales y otras más teóricas, dando soluciones futuras.

3. OBJETIVOS E HIPÓTESIS

3.1 Hipótesis

Los grandes modelos de lenguaje (LLMs) basados en la arquitectura Transformer, al operar exclusivamente mediante inferencia inductiva estadística con parámetros fijos tras el entrenamiento, carecen estructuralmente de la capacidad de aprendizaje continuo necesaria para desarrollar inteligencia artificial general. Esta limitación fundamental no puede resolverse mediante escalado cuantitativo, sino que requiere innovaciones arquitectónicas radicales basadas en la neuroplasticidad biológica.

Específicamente, se plantea que: Las técnicas actuales de adaptación (LoRA, EWC, MANNs) funcionan como módulos de memoria externa sin modificar los parámetros fundamentales del modelo, evidenciando la ausencia de plasticidad real, además del fenómeno del olvido catastrófico que sigue ocurriendo incluso con las nuevas técnicas.

3.2 Objetivos del Proyecto

Explicar el funcionamiento de los Transformers y otros algoritmos para demostrar al público inexperto la falta de inteligencia en los actuales modelos todo ello inicialmente desde un marco filosófico. Demostrar empíricamente las restricciones estructurales de los LLMs actuales respecto al aprendizaje continuo y analizar críticamente si el paradigma predominante puede conducir a inteligencia artificial general. Medir el grado de modificación de los parámetros base de un modelo mediante una métrica propia tras aplicar técnicas de adaptación como LoRA, comparar el rendimiento de modelos con LoRA frente a modelos completamente reentrenados, evaluar tanto la adquisición de nuevas capacidades como la retención de conocimientos previos, los costes computacionales acumulativos de técnicas modulares de memoria externa, proyectar su escalabilidad a largo plazo para aprendizaje continuo real y proponer distintas direcciones de investigación en base a los resultados obtenidos.

3.3 Ámbito, Alcance y Límites

Se realiza un análisis teórico de los fundamentos matemáticos y lógicos de distintos tipos de inferencia en IA, acompañado de experimentación empírica con modelos de escala intermedia que permitan ilustrar principios generalizables. Incluye además una revisión crítica de las técnicas de adaptación, junto con propuestas conceptuales de posibles direcciones alternativas de investigación. No se busca diseñar arquitecturas completamente nuevas con plasticidad real debido a limitaciones de recursos, y los experimentos se restringen a modelos accesibles, extrapolando resultados a escalas mayores. El estudio no aborda cuestiones de consciencia o fenomenología subjetiva, sino que se concentra en las capacidades funcionales de aprendizaje y razonamiento, planteando futuras líneas de investigación desde una perspectiva conceptual y programática, sin implementaciones completas.

4. FUNDAMENTOS TEÓRICOS

4.1 La Inteligencia desde la filosofía

A lo largo de la historia ha habido un gran número de intentos para definir qué es la inteligencia y que es lo que la forma. Dentro de la filosofía podemos encontrar diferentes ramas que definen a la inteligencia de distintas maneras. Una de las más importantes y que

tuvo un gran impacto alrededor de los años 80 fué la creada por Aristóteles, en ella se establece que la inteligencia está formada por un conjunto de reglas que se aplican al mundo real de una u otra manera. Originalmente se crearon dos tipos de razonamiento, deducción e inducción, pero posteriormente fue necesaria la incorporación de una tercera, la abducción que se encarga de describir los procesos más abstractos y sin explicación ya que dependen de muchos factores, algunos de ellos relacionados con la multimodalidad y el aprendizaje continuo.

4.1.1 Deducción

Consiste en la aplicación de un conjunto de reglas que no cambia y está basado en el conocimiento del mundo. Estas relaciones entre conceptos requieren un mínimo de información de cómo funciona el mundo. Hay diferentes tipos de proposiciones, al combinarlas se llega a una conclusión que se supone inteligente.

Los primeros modelos informáticos que pretendieron emular la inteligencia se basaron en este concepto e intentaron utilizar proposiciones lógicas y las diversas reglas dentro de este campo para emular la inteligencia humana, aunque no tardaron en encontrarse con varios problemas. (Ver ejemplo técnico en Anexo A.1)

Este tipo de lógica proposicional utiliza una serie de reglas cerradas que hacen que el sistema no falle, por ejemplo se utiliza para crear la mayor parte del software de los ordenadores que en su nivel más básico utiliza 0s y 1s relacionados por proposiciones lógicas. Los modelos de IA basados en la deducción no aprenden nada, ya que no incorporan reglas nuevas, se mueven en un sistema cerrado y es casi imposible introducir en ellos toda la información sobre el mundo manualmente ya que existen infinidad de factores de los que nosotros a veces no somos conscientes y asumimos. Además, más datos no se correlacionan con un mejor razonamiento.

4.1.2 Inducción

Consiste, en su definición más acorde con la IA actual, en la creación de conocimiento a través de la recopilación de patrones, en resumen el uso de la estadística para predecir estados o en el caso de los LLMs palabras futuras. (Ver ejemplo técnico en Anexo A.2)

Al depender de los datos observados la inducción es un método falible, por ejemplo tras observar a los conductores de una región de Europa un modelo de conducción automática puede predecir distintos tiempos de reacción cuando un peatón cruza un paso de cebra, sin embargo si este modelo se utiliza en otra parte donde los peatones cruzan corriendo o ni siquiera hay pasos de cebra el modelo va a empezar a dar fallos y puede llegar a producir consecuencias catastróficas. Estos modelos se pueden tunear y adaptar a condiciones nuevas pero el aprendizaje automático no aprende en el momento, de manera continua y requiere de una cantidad ingente de datos que no se da en un solo escenario, al contrario, los humanos o seres vivos aprendemos en un periodo de tiempo mucho menor y nos adaptamos mucho más rápido con una cantidad de "datos" muchísimo menor ya que no dependemos completamente de este tipo de inferencia, y poseemos una estructura muy diferente a los parámetros estáticos y conexiones fijas de los LLMs. En resumen somos capaces de incorporar nuevo conocimiento en cualquier momento.

Restricción empírica de los modelos basados en datos

(Ver descripción técnica del funcionamiento en Anexo A.3)

Aunque la IA moderna, particularmente el aprendizaje profundo, es buena detectando patrones y regularidades estadísticas en grandes volúmenes de datos (un proceso inductivo), este enfoque tiene una restricción empírica fundamental: se limita a lo que puede extraer de la "sintaxis" de los datos durante el entrenamiento, sin captar conceptos abstractos o el "significado" más allá de las palabras. Esto da lugar a diversos problemas como: falta de comprensión contextual, vulnerabilidad a "cambios inocuos" como las alteraciones en las imágenes, dificultad con la correferencia que ocurre con menciones indirectas o el uso de pronombres, todo esto se relaciona con la falta de multimodalidad.

La verdadera inteligencia requiere la capacidad de conectar con la realidad y entender el contexto, este trabajo busca demostrar como uno de los caminos hacia una IA más adaptable y que pueda llegar a relacionar y entender el mundo pasan por el desarrollo de nuevas tecnologías.

Los LLMs y el enfoque actual con parámetros estáticos en la innovación

La fé ciega promovida en pos de una IAG que está al caer y que solo necesita de los datos para crear nuevas teorías y ciencia ha sido demostrada inútil en diversas ocasiones, y la

continuación del enfoque con bases en estructuras fijas y predecibles se ha mantenido debido a el interés de potentes corporaciones que han enfocado sus esfuerzos y dinero en la creación de modelos de IA (o LLMs) lo más rápido posible en vez de apoyar ideas individuales que son imprevisibles y no dan tanto beneficio a la empresa.

Un claro ejemplo de todo esto es el Proyecto del Cerebro Humano, un proyecto en el que con ayuda del big data, o cantidades enormes de datos del cerebro como impulsos eléctricos se pretendía simular el funcionamiento del cerebro humano miles de veces y de ahí extraer hipótesis y nuevas teorías acerca del funcionamiento del cerebro. Tras un tiempo diversos neurocientíficos criticaron la iniciativa ya que concluyeron que las simulaciones no iban a conducir a una nueva teoría del funcionamiento del cerebro humano y acertaron ya que el proyecto falló y tuvo que reorientarse hacia el campo de la ingeniería de software. Este es un claro ejemplo de que es necesaria la creación de una idea o teoría antes del análisis de esta con datos para corroborar, esto ocurre en otros campos de la ciencia como la física de partículas en la que campos como el Higgs fueron demostrados a posteriori de su invención teórica.

La IA de la actualidad se basa en el análisis de datos y patrones que recrea lo que existe en el mundo que nosotros conocemos pero esta simulación no tiene porqué llevar a la creación de nuevas teorías. El claro enfoque de diversas corporaciones en este enfoque estadístico podría estar dañando la creación de nuevas arquitecturas.

Otros como Alpha Fold tampoco suponen un avance teórico ya que no da ninguna pista de cómo ocurre el plegamiento de las proteínas que es la parte teórica que falta y en un tercio de los casos la exactitud no es lo suficientemente alta.

Por lo que los modelos basados en big data y algoritmos estadísticos no van a producir ninguna inferencia de abducción o lo que es lo mismo una nueva teoría a partir de los datos por sí mismos.

El sobreajuste es otro problema del big data ya que determinados investigadores confían ciegamente en los datos sin tener una teoría previa y crean teorías muy centradas y calculadas a partir de los datos que no son extrapolables y no funcionan en otros casos. (Ver ejemplo técnico en Anexo A.4)

(Ver caso de estudio: predicción de terremotos en Anexo A.5)

El aprendizaje automático se da solo con inducción, así que los investigadores de la disciplina deberían mostrarse más escépticos de lo que son habitualmente sobre las perspectivas de la inteligencia artificial general.

La Regularidad

La inducción permite a la inteligencia comportarse como un detector de regularidad. La IA estadística destaca a la hora de capturar regularidades a partir del análisis de datos, y ese es el motivo por el que las labores de reconocimiento visual de objetos, como la identificación de fotos de rostros humanos y mascotas, se cuentan entre sus éxitos. Los píxeles de un rostro se pueden distribuir y regular de manera que sean estudiados y clasificados. Sin embargo, puesto que esos sistemas aprenden a partir de las observaciones de unos patrones de impulso específicos, tienen problemas de fragilidad. Tal y como han señalado Gary Marcus, Ernest Davis y otros investigadores, incluso cambios en apariencia benignos como pasar el color de fondo del blanco al azul en las tareas de detección de objetos pueden llevar a que su rendimiento se degrade. La persona ignorará sin problemas que se añadan unas letras sin importancia en el lado de la imagen, pero los modelos fallan con resultados aleatorios fruto de los cálculos que incorporan los píxeles modificados. En el ámbito del lenguaje o la generación de contenido, debido a su inherente dependencia en los datos, han llegado a ocurrir problemas de sesgo que han sido solucionados simplemente eliminando los datos manualmente, esto ha ocurrido con modelos chinos en cuestiones políticas o en modelos que contenían sesgos racistas o xenófobos.

Hay que destacar una observación del pionero de la IA Yoshua Bengio según la cual las redes neuronales profundas «tienden a aprender regularidades estadísticas en el conjunto de datos en vez de conceptos abstractos de nivel superior». La inteligencia humana se reduce a la función de "reconocedores de patrones" organizados jerárquicamente, una hipótesis que es "interesante" pero que asume una simplicidad en la inteligencia humana que no es real (como expresa el científico de la IA y el ML Erik J. Larson).

Sin embargo hay que anotar un tanto para la IA basada en big data y este está relacionado con sectores que se benefician de los datos y que en la actualidad están explosionando, muchos de ellos están relacionados con el ámbito económico o de empresa, incluso ayuda en la optimización en muchos procesos que se hacen más económicos o beneficiosos para la sociedad e incluso el medioambiente.

4.1.3 Abducción

Pierce (lógico, filósofo y científico americano), describe la abducción como una deducción rota. (Ver formulación lógica en Anexo A.6)

Esto es considerado una falacia llamada la afirmación del consecuente. Esto es una suposición que para nosotros es correcta pero no lo es desde la lógica por lo que Pierce la consideró una manera sencilla de expresar como la abducción no es igual que a la deducción o la inducción.

La Abducción como Motor del Pensamiento Real

A diferencia de la deducción, la abducción no preserva la verdad de las premisas a la conclusión, lo que significa que la conclusión es solo una posible explicación, no una certeza lógica. Esto subraya que la abducción es fundamentalmente diferente y no una forma extendida de deducción, ya que su esencia (la conjetura) "quebranta esa voluntad de preservar la verdad" inherente a la inferencia deductiva, la abducción se centra en lo que "podría ser", es decir, en la formulación de las mejores explicaciones o hipótesis para fenómenos observados. En la vida real, la abducción sucede por ejemplo en las imaginaciones con de objetos que no están en el conjunto de cosas que nosotros observamos, contrafactuales o suposiciones de hechos que podrían haber ocurrido pero no lo han hecho, teorías que todavía no están demostradas y pueden o no ser correctas. Es la capacidad de generar conjeturas y revisarlas constantemente es fundamental para el razonamiento humano, la comprensión del lenguaje natural y la adaptación en el mundo real que ocurre gracias a la neuroplasticidad y el aprendizaje continuo.

Estos tres tipos de inferencia han sido creados para establecer las diferentes maneras y límites por los que nosotros como humanos generalmente creamos o utilizamos nuevo conocimiento. Cada uno está separado de los otros y no pueden confundirse. Lo que nos falta para poder llegar a una IA mucho más humana o lo que nosotros entendemos por real, es una teoría del funcionamiento de lo que he descrito como abducción.

Limitaciones de la IA Actual (Frente a la Abducción)

Falta de Sentido Común y Contexto: La IA tiene dificultades para realizar inferencias de sentido común, especialmente cuando se introduce contexto que cambia la interpretación lógica. Esto se debe a que la IA procesa datos de manera superficial, sin un "armazón conceptual" profundo que le permita entender las relaciones y el significado real.

A pesar de décadas de trabajo, la IA sigue sin poder representar el conocimiento del sentido común de manera suficiente para actuar en escenarios del mundo real o comprender el lenguaje y el razonamiento, simplemente lo imita a la perfección gracias a los millones de ejemplos existentes en el mundo. Por eso es que la proposición de este trabajo es la creación de modelos que funcionen en tiempo real y adapten sus parámetros para poder

incorporar conocimiento y relaciones constantemente lo que podría llevar a una inteligencia algo más verdadera ya que se relacionarían diferentes estímulos en tiempo real.

4.2 Modelos de ML

Transformers

En los modelos conocidos como LLMs nos encontramos con una nueva arquitectura presentada en 2017 en el paper "Attention is all you need" que revolucionó la IA relacionada con el lenguaje natural.

Un ejemplo de esto es la creación y el auge de modelos como Chat-GPT que literalmente significa Generative Pre Trained Transformer, lo que en español significa que es un modelo generativo entrenado previamente con la base de un Transformer. El output final de estos tipos de modelos consiste en un arco de probabilidades para la siguiente palabra a la que ya se ha generado, por ejemplo: Ayer me compré un > coche (30%), perro (12%), abanico (3%) etc. Estos modelos trabajan con tokens que son divisiones del texto y las palabras en fragmentos más pequeños y más óptimos.

(Ver proceso técnico de generación en Anexo B.1)

GPT - 3 contiene 175.181.291.520 parámetros. Que están mayoritariamente en forma de matrices o tensores que contienen los "pesos" de este tipo de modelos.

Como he mencionado anteriormente estos modelos trabajan con tokens que son divisiones de las palabras, todas estas subpalabras se almacena en un vocabulario con el que el modelo trabaja, la ventaja de los tokens es que son más eficientes que usar solo letras, siguen pudiendo usarse en distintos idiomas y optimizan el entrenamiento mucho más que otros métodos como las palabras ya que se necesita un menor vocabulario.

(Ver representación vectorial de tokens en Anexo B.2)

(Ver teorema de Johnson-Lindenstrauss y dimensionalidad en Anexo B.3)

En GPT - 3 los vectores de los tokens tenían 12.288 dimensiones o coordenadas, y el modelo contaba con 50.257 tokens diferentes.

Funcionamiento (Paper)



Anteriormente los modelos que intentaban dominar la traducción y el uso del lenguaje natural eran de carácter recurrente, o redes convolucionales que incluían un codificador y un decodificador, los mejores modelos usaban un tipo de atención que relacionaba los tokens entre sí. En el nuevo paper se presentó un tipo de modelo que prescindía de la recurrencia y las convoluciones y se focalizaba solo en los mecanismos de atención.

Los modelos anteriores al ser recurrentes tenían una naturaleza lineal o secuencial lo que limitaba su capacidad en secuencias de palabras o tokens más largas, incluso con nuevos trucos que optimizaban las computaciones su naturaleza seguía limitando un desarrollo más agresivo y con futuro. Los mecanismos de atención funcionan mucho mejor en tareas de carácter secuencial y no importa la longitud de las secuencias o la distancia entre el input y el output. Los Transformers son un tipo de modelos que se basan totalmente en mecanismos de atención y consiguen paralelizar la carga de trabajo mucho mejor.

Estructura

El modelo consta de un codificador y un decodificador, el codificador convierte una secuencia del input de tokens a una representación en un espacio de x dimensiones o de manera más simple convierte cada palabra o token en su correspondiente vector de un espacio con x dimensiones como hemos mencionado anteriormente. El decodificador por definición se encarga de convertir los vectores resultantes en tokens o palabras que se añaden al output y además se vuelven a introducir en el modelo para seguir generando una respuesta.

Los transformers utilizan esta estructura con varias capas en el medio para calcular los vectores resultantes, el codificador cuenta con 6 capas (aunque es variante) y cada una de ellas tiene dos subcapas, la primera subcapa tiene un mecanismo de multi-head attention y la segunda subcapa es una simple feed-forward network que es la típica con forma de perceptrón multicapa.

(Ver detalles técnicos de conexiones residuales y normalización en Anexo B.4)

El decodificador también cuenta con 6 capas (variantes también), idénticas a las del codificador pero añadiendo una 3ra subcapa que utiliza el multi-head attention mechanism en el output del codificador.

La otra subcapa que también utiliza el mecanismo de multi-head attention, tiene una modificación para prevenir que durante el entrenamiento el modelo preste atención a las palabras que se supone que tiene que predecir.

(Ver funcionamiento detallado del decodificador en Anexo B.5)

El mecanismo de atención

(Ver fórmula técnica en Anexo B.6)

Se basa en las operaciones con vectores y matrices que representan y conectan los tokens de una manera estadística, hay tres tipos de vectores muy importantes: query, keys y values con los que se obtiene el output. El mecanismo de atención usado inicialmente se llama producto vectorial escalado. Aquí es donde las queries de un token se comparan con las keys de todos los demás tokens para calcular cuánto peso ponerle a cada value.

(Ver descripción técnica de Q, K y V en Anexo B.7)

(Ver proceso paso a paso del mecanismo de atención en Anexo B.8)

(Ver cálculo de matrices Q, K y V en Anexo B.9)

(Ver ejemplo práctico: "Yo no como manzanas" en Anexo B.10)

En conclusión lo que sacamos de estas operaciones es fundamentalmente una transformación del espacio de embeddings para una frase (o secuencia de tokens) en particular. Su propósito principal es mejorar las representaciones de cada token al capturar y expresar de manera más efectiva las relaciones específicas que tiene con todos los demás tokens dentro de ese contexto dado.

Al unir la atención (los pesos de relevancia de Q y K) con los rasgos potenciados (los vectores V), el Transformer genera una nueva representación contextualizada para cada token. Esta nueva representación encapsula no solo el significado original del token, sino también: Qué otros tokens son más relevantes en ese contexto específico (determinado por Q y K). Cuáles son los rasgos más importantes que esos tokens relevantes aportan (filtrados y potenciados por sus respectivos WV matrices).

El resultado final no es solo una lista de "las palabras más relacionadas", sino un vector único para cada palabra que ahora incluye y destaca los rasgos que la conectan con su entorno en esa frase particular.

Multi-head

Es más preciso y beneficioso si este proceso se lleva a cabo en paralelo con diferentes proyecciones de Q, K y V ya que se "aprenden" o encuentran más patrones estadísticos entre los tokens o palabras. Específicamente se crean distintos subespacios transformados en los que las palabras o tokens se relacionan de una manera diferente.

En el modelo original se incorporaron 8 cabezas diferentes o capas de atención (Chat-GPT3 tenía 96 y, que tenían sus dimensiones reducidas de la manera $dK = dV = d(\text{modelo})/8 = 64$ por lo que se repartían las dimensiones entre las cabezas para que el coste computacional fuese igual al de una sola cabeza de mayor dimensión.

Uso de la atención

En los Transformers, tanto en las capas codificadoras como en las decodificadoras encontramos subcapas que utilizan el mecanismo de atención.

En las subcapas del codificador, las queries, keys y values provienen todas de la misma entrada (es decir, se realiza self-attention dentro de la misma secuencia).

En cambio, en las subcapas del decodificador, hay dos tipos de atención: En la primera subcapa, se realiza self-attention con enmascaramiento (masked self-attention), donde queries, keys y values también vienen de la misma entrada pero se impide que un token vea a los futuros. En la segunda subcapa, se aplica encoder-decoder attention, donde las queries vienen del decodificador y las keys y values vienen del output del codificador.

Esta estructura permite que el decodificador se apoye tanto en el contexto previo generado como en la información codificada de la entrada original.

Como he mencionado anteriormente aparte de las subcapas que utilizan el mecanismo de atención el modelo también contiene subcapas de fully connected feedforward networks que tienen la estructura similar (aunque mucho más simple) a la de un perceptrón multicapa o la típica red neuronal, estas subcapas lo único que llevan a cabo son transformaciones para

cada token en específico sin cambiar las relaciones previamente establecidas, ayudan a mejorar la representación de cada token una vez que ya se ha aplicado la atención.

(Ver fórmula de la red feedforward en Anexo B.11)

Codificación posicional

Cuando se convierte un token a un vector en la capa codificadora y en la decodificadora se modifica el vector añadiendo información acerca de su posición en la frase, esto se puede hacer de manera fija o variable.

(Ver aclaración sobre la codificación posicional en Anexo B.12)

(Ver aclaración sobre atención vs auto-atención en Anexo B.13)

Conclusiones transformers

Al final, cada token tiene una representación contextualizada que se pasa por una capa final lineal para transformarse en una distribución de probabilidad sobre todo el vocabulario. Durante el entrenamiento, esta salida se compara con el token real siguiente y se usa la retropropagación para ajustar los pesos del modelo —incluyendo las capas de atención y feedforward— para minimizar esa pérdida.

Una vez entrenado, el modelo puede utilizarse para generar texto prediciendo el token más probable a continuación, en función de los tokens anteriores. Esto se hace paso a paso, alimentando el modelo con su propia salida previa, a través de este proceso algo interesante que ocurre es que los vectores que representan a los tokens van cambiando dependiendo de los tokens de entrada (contexto). Este proceso continúa hasta que se genera un número máximo de tokens, o hasta que aparece un token de parada. La longitud máxima de entrada y de generación está limitada por la ventana de contexto, que es una restricción programada previamente en el modelo.

(Ver ejemplo completo del proceso de generación en Anexo B.14)

Los Transformers y más específicamente la auto-atención son mucho más óptimos en cuanto a la computación de secuencias de carácter largo, específicamente los primeros modelos se entrenaron con 4.5 millones de frases, 37000 tokens en GPUs Nvidia P100 y tomaron desde 12 horas hasta 3.5 días de entrenamiento.

En el paper original de los Transformers, los autores destacaron que diferentes cabezas de atención aprendían espontáneamente a reconocer patrones estructurales y semánticos dentro de las frases, como por ejemplo relaciones entre sujetos y verbos o referencias pronominales. Esto les resultaba especialmente interesante porque el modelo no fue programado explícitamente para entender la gramática, sino que estos patrones emergieron como resultado del entrenamiento sobre grandes cantidades de texto.

(Ver funcionamiento del proceso de multi-atención en Anexo B.15)

Esta capacidad es completamente normal ya que nosotros como humanos en nuestro lenguaje utilizamos estas estructuras que se repiten tantas veces que al analizarlas estadísticamente acaban apareciendo. Por lo que los famosos LLMs que están detrás de la mayoría de modelos de IA en la actualidad expulsan texto que sigue patrones estadísticos del texto del que ha sido entrenado, y una vez entrenados sus parámetros o matrices no se actualizan.

4.3 Otros modelos y tipos de machine learning

Perceptrón

El perceptrón fué el primer intento de replicar el funcionamiento del cerebro humano y por tanto su inteligencia, la idea apareció en 1943 y sus creadores fueron Warren McCulloch y Walter Pitts inspirados por la lógica de Turing y Russell. La creación de McCulloch y Pitts era meramente teórica y allanaba el camino para el primer modelo de red neuronal.

Un perceptrón es un modelo de red neuronal simple que se utiliza para la clasificación binaria y consistía en un conjunto de operaciones que daban lugar a un output, las operaciones según los autores se asemejaban a las conexiones neuronales y el valor de los pesos que son los valores con los que se realizan las operaciones se van ajustando para tener el output deseado. El output originalmente consistía en que al final consiguieses una activación representada con un 1 o nada representado con un 0.

(Ver estructura técnica del perceptrón en Anexo C.1)

Después de la propuesta teórica de McCulloch y Pitts, Rosenblatt desarrolló el Mark I Perceptrón, el primer modelo físico capaz de aprender de los datos. A diferencia de los

modelos deductivos anteriores, este perceptrón ajustaba sus pesos y sesgos para separar linealmente conjuntos de datos y clasificar nuevos ejemplos: fue el paso de la deducción lógica a la inducción estadística, aunque con limitaciones.

(Ver ejemplo de aplicación: clasificación obeso/no obeso en Anexo C.2)

(Ver funcionamiento técnico del entrenamiento y predicción en Anexo C.3)

Resumen del trabajo hecho por McCulloch y Pitts en "A logical calculus of the ideas immanent in nervous activity".

Para McCulloch y Pitts, el hecho de que las neuronas se disparen o no, sin estados intermedios, significaba que su funcionamiento podía ser recreado usando únicamente proposiciones lógicas. En el paper se afirma que la activación de una neurona corresponde a una proposición lógica que resulta verdadera. Sin embargo, esto no es correcto: las neuronas no se comportan de forma tan estrictamente binaria en la realidad.

Ellos presentan dos problemas que posteriormente omiten y no enfrentan: primero, que las redes neuronales reaccionan de manera distinta tras nuevos estímulos; segundo, que estos estímulos alteran tanto la estructura como el umbral de activación de las neuronas —es decir, modifican la cantidad de receptores químicos y eléctricos en las sinapsis que regulan su activación.

No obstante, poco después explican que, para que su modelo funcione, necesitan imponer una serie de reglas que no se ajustan a la realidad biológica. Afirman, por ejemplo, que un número fijo de sinapsis debe ser estimulado para que una neurona se active, y que ese umbral es constante, sin importar el contexto ni la actividad previa. Esto es falso. También sostienen que la estructura de la red es fija, lo cual contradice completamente lo que hoy sabemos: la neuroplasticidad es un componente esencial de las redes neuronales biológicas.

La neuroplasticidad altera las sinapsis (aprendizaje hebbiano), modifica estructuras existentes e incluso da lugar a nuevas neuronas. Frente a esto, McCulloch y Pitts pretenden modelar procesos inteligentes utilizando proposiciones lógicas invariables.

El cálculo lógico que proponen consiste en construir relaciones mediante operaciones lógicas —como AND, OR y NOT— capaces de computar cualquier función booleana bajo reglas fijas. Según ellos, estas estructuras podrían modelar ciertos aspectos de la activación



neuronal. El primer error conceptual de fondo es tratar el cerebro como si fuera un ordenador.

Desde su trabajo, la inteligencia ha sido reducida a operaciones lógicas y computables. Esta decisión técnica —comprensible en su contexto histórico— generó una tradición que ignora aspectos fundamentales de la inteligencia humana: la plasticidad, la experiencia encarnada, la adaptabilidad y la conciencia. Al centrarse en lo que es fácil de modelar —reglas fijas, proposiciones lógicas, funciones deterministas—, la inteligencia artificial ha seguido un camino que simula partes de la inteligencia, pero no la comprende ni la reproduce en su complejidad humana. Esto no es un accidente: es el resultado de un enfoque funcionalista que prioriza la eficiencia sobre la comprensión. Desde sus orígenes, los científicos y desarrolladores han elegido, conscientemente o no, centrarse en los aspectos que les resultaban técnica o computacionalmente convenientes.

Perceptrón multicapa

Tras la creación del perceptrón y sus distintas modificaciones, el ámbito del machine learning fué progresando más hacia un enfoque estadístico que lógico, o desde un punto filosófico pasó de la deducción a la inducción. El primer problema surgió con la compuerta lógica XOR, con una sola capa oculta no era posible resolverlo ya que solo podían aprender separaciones lineales, y el entrenamiento de modelos con más capas no era posible.

Entonces llegaron David E. Rumelhart, Ronald J. Williams y Geoffrey Hinton quienes desarrollaron el algoritmo de retropropagación que hizo posible entrenar a modelos con más capas ocultas y originó lo que hoy se conoce como deep learning o aprendizaje profundo (varias capas). El perceptrón multicapa permitió la creación de hiperplanos en mayores dimensiones y por tanto la separación de más clases de datos **(Ver ejemplo en Anexo D.1)**, por ejemplo reconocer números del 1 al 10 según los píxeles introducidos como un vector, por ejemplo una imagen 28 x 28 te da 784 píxeles que se pasan a las primeras 784 neuronas.

Cada "conexión" entre neuronas tiene asignada un valor llamado "peso" o weight en inglés que se multiplica por el valor de la neurona anterior, estos pesos también están normalizados para que los cálculos no sean computacionalmente costosos.

(Ver función Sigmoide y otros tipos de funciones de activación en Anexo D.2)

(Ver cálculo de pesos y sesgos totales en Anexo D.3)

Ahora entramos en cómo se computa la dirección y cantidad de cambio en los pesos y los sesgos, que ocurre por el algoritmo de retropropagación. Las diferentes épocas de entrenamiento o epoch en inglés utilizan diferentes subconjuntos de los datos. La retropropagación cambia los pesos y sesgos a través de las derivadas parciales que conectan el valor de la última neurona con el peso deseado y así se conoce la relación de cambio que hay entre ambos, tras cada cambio se calcula el coste medio que calcula el error **(Ver función de coste en Anexo D.4)**. Finalmente el objetivo es reducir la cost function al mínimo mediante sucesivas iteraciones, para que el modelo prediga correctamente.

(Ver desarrollo matemático completo de la retropropagación en Anexo D.5)

CNN

Las redes neuronales convolucionales (CNN, por sus siglas en inglés) surgieron para resolver limitaciones que presentaban los perceptrones multicapa (MLP) en el tratamiento de datos visuales. Un MLP, al procesar imágenes píxel a píxel, no tiene en cuenta la estructura espacial y además no es robusto a traslaciones: el mismo objeto, si aparece desplazado en la imagen, se percibe como un patrón completamente distinto. Las CNN reducen este problema gracias al uso de convoluciones, que aplican los mismos filtros a lo largo de toda la imagen, permitiendo reconocer patrones independientemente de su posición, y con el uso de capas de pooling, que introducen cierta invariancia a traslaciones y reducen la dimensionalidad.

El diseño de estas redes estuvo inspirado en los hallazgos de David Hubel y Torsten Wiesel en los años 60 sobre la corteza visual de los mamíferos. Descubrieron que existen neuronas simples, que responden selectivamente a estímulos básicos como bordes u orientaciones, y neuronas complejas, que integran estas respuestas para reconocer formas más elaboradas. Los creadores de las CNN establecieron un paralelismo: las primeras capas de la red extraen características simples (bordes, colores, texturas), mientras que las capas posteriores combinan esas representaciones para identificar estructuras complejas (partes de un objeto, o incluso el objeto completo). **(Ver funcionamiento técnico detallado en Anexo E.1)**

(Ver proceso de entrenamiento supervisado en Anexo E.2)



A lo largo de la historia de modelos como estos siempre se han necesitado capacidades ingentes de imágenes (AlexNet 1.2 mill) para que estadísticamente el modelo reduzca su error, sin embargo un humano no necesita de tantos datos para reconocer objetos.

RNN

Las redes neuronales recurrentes (RNNs) fueron precursoras de los modelos de lenguaje actuales. Su característica principal es que el estado oculto de cada paso temporal depende no solo de la entrada actual, sino también del estado anterior, lo que introduce una forma de memoria secuencial. Gracias a esta retroalimentación, las RNNs pudieron modelar datos en secuencia, como texto o audio. Aunque su memoria era limitada y sufrían problemas de entrenamiento en secuencias largas, variantes como LSTM y GRU mejoraron este aspecto y abrieron el camino hacia los modelos más avanzados como los transformers, base de los LLMs modernos.

Modelos de difusión

(Ver funcionamiento técnico completo en Anexo E.3)

Los modelos de difusión son modelos generativos que crean imágenes de alta calidad a partir de ruido aleatorio mediante un proceso iterativo que combina CNNs y Transformers, representando uno de los avances más significativos en modelos generativos modernos, aunque manteniendo el enfoque clásico de los parámetros estáticos.

Aunque pueden parecer flexibles debido a técnicas como coger aleatoriamente los n tokens/palabras más probables por lo que producen textos variados y coherentes, los LLMs no modifican su estructura interna ni consolidan nuevas conexiones durante la interacción con el usuario. Para actualizar sus capacidades es necesario un nuevo proceso de entrenamiento o ajuste (fine-tuning), que no ocurre de manera espontánea como en el cerebro humano y conlleva pérdida de información.

4.4 Neuroplasticidad y aprendizaje continuo biológico

La neuroplasticidad es la capacidad del cerebro humano de modificar su estructura y funcionamiento en respuesta a la experiencia, el aprendizaje y el entorno que se incorpora gracias a los sentidos. Cada vez que aprendemos algo nuevo, se forman y refuerzan conexiones sinápticas entre neuronas, mientras que aquellas que no se utilizan tienden a

debilitarse o desaparecer. Este proceso dinámico permite que el cerebro optimice sus recursos, conserve lo que resulta útil y se adapte a cambios a lo largo de toda la vida. Gracias a ello, las personas no solo adquieren conocimientos y habilidades en la infancia, sino que también pueden continuar aprendiendo y desarrollándose en la adultez, aunque la plasticidad sea menos intensa que en los primeros años.

El aprendizaje continuo depende directamente de esta plasticidad. Practicar un idioma, tocar un instrumento, desarrollar una nueva habilidad o incluso modificar hábitos de comportamiento implica que el cerebro reorganice sus redes neuronales, fortaleciendo rutas de comunicación y generando nuevas conexiones, además la adaptabilidad constante que ofrece la neuroplasticidad permite que se establezcan relaciones entre diferentes inputs constantemente lo que da lugar al aprendizaje relacional.

(Ver explicación completa en ANEXO F: Neuroplasticidad vs. IA - Comparación Cuantitativa)

Resumen Teórico

En los modelos actuales de modelado del lenguaje, que es uno de los pilares de la psicología y la inteligencia humana, nos encontramos con la particular restricción de la falta de aprendizaje continuo debido a parámetros fijos tras el entrenamiento. Los Transformers que son la base de estos modelos del lenguaje, tienen una posición dominante en el desarrollo actual. En general en el enfoque actual en los modelos generativos, ya sean de imágenes, sonido, video, ambos o incluso los nuevos llamados LAMs que son en palabras más simples "agentes de IA", se utilizan grandes cantidades de datos que se relacionan estadísticamente entre ellos. El enfoque actual se encuentra con los siguientes desafíos: multimodalidad completa en tiempo real que ayude a la exploración de modelos que aprendan a establecer relaciones entre conceptos y objetos del mundo real, memoria a largo y corto plazo para cualquier ocasión o usuario, costes computacionales, de recursos, y muchos otros.

En resumen, mientras que los seres humanos aprenden de forma continua y adaptativa gracias a la neuroplasticidad, los LLMs se mantienen estáticos tras su entrenamiento inicial, esto marca una diferencia fundamental.



Existen diversos trucos y excepciones por los que sí se alteran, pero ello conlleva varios problemas. En tiempos recientes se ha desarrollado la idea de almacenar datos pasados, conversaciones e información en bases de datos o espacios latentes nuevos en los que el modelo cada vez que interactúa con el usuario "busca". Las redes neuronales con memoria aumentada son una arquitectura prometedora que combina redes neuronales con almacenamiento de memoria externa. Al procesar secuencias de entrada, como las indicaciones del usuario, las MANN pueden leer y escribir en la memoria. Muchas utilizan mecanismos de atención para aislar los componentes de memoria más relevantes para cada tarea. La memoria episódica de gradiente (GEM) es un ejemplo de MANN que permite a los modelos de IA almacenar y recordar experiencias pasadas para fundamentar nuevas tareas y preservar el conocimiento adquirido previamente. Estos espacios de almacenamiento funcionan pero conllevan varios problemas, el primero de ellos siendo el más obvio y el punto central de mi experimentación, los parámetros no cambian lo que quiere decir que el "conocimiento" o relaciones entrenadas son siempre las mismas.

Otra idea muy potente es la de alterar el menor número de pesos posibles y que afecten en menor medida al rendimiento del modelo para incorporar nuevos patrones al reentrenar en datos más específicos, la consolidación elástica de pesos (EWC) es una de estas técnicas que añade una penalización a la función de pérdida por ajustes en los pesos del modelo importantes para tareas antiguas. La inteligencia sináptica funciona de forma similar, desincentivando al modelo a cambiar parámetros importantes. Ambas técnicas reducen la probabilidad de que el modelo pierda información previa, el problema aquí yace en que los parámetros alterados siempre pierden algo de información previa y en general ocurre el catastrophic forgetting que en resumen demuestra una pérdida de información con respecto al estado pasado del modelo en cierta área. En resumen, operan dentro del número de parámetros existentes para retardar el olvido o preservar pesos importantes, en lugar de adaptarse a un nivel físico creando nuevos parámetros. El olvido catastrófico se reduce pero no se resuelve por completo porque los parámetros originales de la red son limitados y solo ocurren a un nivel informático que supone un coste computacional mucho mayor que en los parámetros/conexiones físicas.

Observado por primera vez por Michael McCloskey y Neal J. Cohen en 1989, el olvido catastrófico se produce como resultado de la forma en que los algoritmos de aprendizaje automático se adaptan a nuevos conjuntos de datos. El proceso de entrenamiento de

modelos de aprendizaje profundo, como los modelos de lenguaje grandes (LLM), implica exponer el modelo a los datos y permitir que actualice sus ponderaciones en consecuencia. Un artículo de 2023 descubrió que afecta a los modelos grandes con mayor gravedad que a los pequeños¹.

En tiempos recientes se ha profundizado en la creación de modelos que imiten la neuroplasticidad humana y añadan algunos parámetros extra al modelo ya existente sin alterar los parámetros del LLM ya existente que se "congela", esto ocurre con los LoRAs (Adaptación de Bajo Rango) que son el enfoque estudiado experimentalmente en este proyecto.

6. METODOLOGÍA Y FUENTES

A lo largo de los dos pasados años he estado estudiando e investigando acerca de los diferentes algoritmos, modelos e ideas que se han desarrollado para crear lo que es conocido como inteligencia artificial. El trabajo tuvo distintas fases, investigación, pruebas y conclusiones. Dentro de ellas se podría decir que hay muchas más subfases. La investigación se hizo a través de libros académicos y de divulgación, personajes públicos que explican diferentes modelos de machine learning a través de sus distintas plataformas, artículos científicos de acceso público, y la propia experimentación con algoritmos para ver cómo funcionan. Todas las fuentes han sido fielmente contrastadas y aceptadas por la comunidad científica.

7. EXPERIMENTACIÓN Y ANÁLISIS DE RESULTADOS

Para llevar a cabo el experimento creé una métrica (SPC, similitud paramétrica conceptual) que compara el nivel de similitud entre los parámetros del modelo principal antes y después del entrenamiento del LoRA, además se incluye una comparación del rendimiento en tareas antiguas del modelo + LoRA contra el del modelo si se reentrena completamente con la nueva información para evidenciar que el cambio de los parámetros principales lleva al olvido catastrófico. Los resultados son los siguientes:

Los resultados del experimento confirman que LoRA no introduce plasticidad real en los parámetros del modelo base, ya que los pesos originales permanecen inalterados (SPC $\approx$ 1.0; 100% de capas idénticas bit a bit). Esto evidencia que LoRA funciona como un módulo

¹ Yun Luo et al., "Under Review an Empirical Study of Catastrophic Forgetting in Large Language Models during Continual Fine-Tuning," n.d. <https://arxiv.org/pdf/2308.08747>

de memoria externa acoplada, capaz de aprender nuevas tareas sin alterar la estructura interna del modelo.

A nivel funcional, LoRA alcanza una reducción del *loss* del 99.6%, logrando un rendimiento equiparable al reentrenamiento completo y mitigando ligeramente mejor el olvido catastrófico (+1.35%). Estos resultados demuestran su eficacia en escenarios de aprendizaje incremental modular o multitarea controlada, donde cada tarea puede asociarse a un conjunto independiente de adaptadores.

En términos de coste computacional, LoRA presenta un entrenamiento inicial mucho más eficiente, al requerir la actualización de solo una pequeña fracción de los parámetros del modelo ($\approx 0.1-1\%$). Esto reduce drásticamente el consumo de GPU, memoria y energía durante el entrenamiento de una tarea individual. Sin embargo, si LoRA se emplea para aprendizaje continuo, el coste acumulativo se incrementa significativamente, ya que cada nueva tarea demanda el almacenamiento y gestión de un nuevo conjunto de adaptadores. A largo plazo, esta acumulación de módulos puede superar el tamaño y la complejidad del modelo base, afectando la escalabilidad y el rendimiento en inferencia.

Por tanto, aunque LoRA ofrece una alternativa eficiente para extender modelos pre entrenados mediante memoria externa y modularidad, su naturaleza no plástica a un nivel físico y su creciente coste acumulativo limitan su viabilidad para el aprendizaje continuo real, donde se requiere consolidar y optimizar el conocimiento dentro de una única arquitectura dinámica. Además conviene añadir que aún requiere de infinidad de ejemplos por tarea para un entrenamiento efectivo (no es un entrenamiento único), no reorganiza la red de forma autónoma ni reasigna la capacidad dinámicamente, si se añaden demasiadas tareas, la memoria y el rendimiento pueden verse afectados, ya que los recursos computacionales crecen exponencialmente. **(Ver anexo G, Gráficas y Datos)**

8. CONCLUSIONES

8.1. Evaluación

A la vista de las actuales limitaciones en el campo de la IA uno de los caminos hacia la abducción que se proponen son los parámetros/ conexiones adaptables, ya que para comprender el lenguaje, realmente hace falta estar en contacto con el mundo, en otras palabras: multimodalidad, y esto tiene que ocurrir de manera continua obteniendo inputs de

distintos tipos al mismo tiempo lo que podría dar lugar a un tipo de aprendizaje relacional como el que ocurre en los seres humanos. La estructura de los modelos actuales se topa con multitud de limitaciones en el aspecto del aprendizaje continuo, varias de las cuales han sido expuestas teóricamente y prácticamente a lo largo de este trabajo, estas limitaciones apoyan la hipótesis de la necesidad de estructuras y parámetros adaptables más allá de la estadística y las placas de silicio.

8.2 Divulgación

Código qr

8.3 Futuras líneas de investigación

A partir de los hallazgos de este trabajo, se proponen las siguientes direcciones de investigación:

Arquitecturas con plasticidad paramétrica real: Explorar diseños que permitan la creación, eliminación y modificación dinámica de parámetros durante la inferencia, inspirados en mecanismos de neurogénesis y poda sináptica biológica.

Consolidación selectiva de conocimiento: Desarrollar algoritmos que identifiquen qué información debe integrarse permanentemente en los parámetros base y cuál puede mantenerse en módulos temporales, emulando los procesos de consolidación de memoria a largo plazo.

Aprendizaje multimodal continuo: Investigar arquitecturas que procesen simultáneamente múltiples modalidades sensoriales (texto, imagen, audio) en tiempo real, estableciendo relaciones cross-modales de forma autónoma sin reentrenamiento.

Métricas de plasticidad: Refinar y validar métricas como la SPC propuesta, desarrollando estándares cuantitativos para evaluar la verdadera capacidad de aprendizaje continuo en sistemas de IA.

Hardware especializado: Explorar sustratos computacionales alternativos (computación neuromórfica, sistemas analógicos) que faciliten la implementación eficiente de plasticidad paramétrica a nivel físico, reduciendo los costes computacionales del aprendizaje continuo.

9. REFERENCIAS CONSULTADAS (bibliografía, páginas Web, etc.)

Libros

Ananthaswamy, A. (s.f.). *Why machines learn*. Penguin.

Goodfellow, I., Bengio, Y., & Courville, A. (s.f.). *Deep Learning*. Recuperado de <https://www.deeplearningbook.org/>

Larson, E. J. (s.f.). *The Myth of Artificial Intelligence: Why Computers Can't Think the Way We Do*. Harvard University Press.

Nielsen, M. A. (s.f.). *Neural networks and deep learning*. Recuperado de <http://neuralnetworksanddeeplearning.com/>

Artículos Académicos

Abernot, M., Azemard, N., & Todri-Sanial, A. (2023). Oscillatory neural network learning for pattern recognition: an on-chip learning perspective and implementation. *Frontiers in Neuroscience*, 17. <https://doi.org/10.3389/fnins.2023.1196796>

Azcona, M. (2019). Abducción e inferencia a la mejor explicación: criterios para su delimitación metodológica. *Epistemología e Historia de la Ciencia*, 4, 35-55.

Luo, Y., Yang, Z., Meng, F., Li, Y., Zhou, J., & Zhang, Y. (2023, 17 de agosto). An Empirical Study of Catastrophic Forgetting in Large Language Models During Continual Fine-tuning. *arXiv.Org*. <https://arxiv.org/abs/2308.08747>

McCulloch, W. S., & Pitts, W. (s.f.). A logical calculus of the ideas immanent in nervous activity. University of Illinois, College of Medicine, Department of Psychiatry at the Illinois Neuropsychiatric Institute, University of Chicago.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017, June 12). *Attention Is All You Need*. arXiv.Org. <https://arxiv.org/abs/1706.03762>

Large Concept Models: Language Modeling in a Sentence Representation Space. (n.d.). Research - AI at Meta. Retrieved October 12, 2025, from <https://ai.meta.com/research/publications/large-concept-models-language-modeling-in-a-sentence-representation-space/>

Artículos en Línea y Blogs

Abhijithcprakash. (2024, 22 de febrero). Building Your Own Text Generation Model: A Beginner's Guide. *Medium*. <https://medium.com/@abhijithcprakash/building-your-own-text-generation-model-a-beginners-guide-5261fad1e6bc>

Haque, Kh. N. (2023, 1 de febrero). What is Convolutional Neural Network — CNN (Deep Learning). *Medium*.
<https://nafizshahriar.medium.com/what-is-convolutional-neural-network-cnn-deep-learning-b3921bdd82d5>

Mullin, E. (1970, 1 de enero). El fracaso de los proyectos de la UE y EEUU para entender el cerebro. *MIT Technology Review en español*.
<https://technologyreview.es/article/el-fracaso-de-los-proyectos-de-la-ue-y-eeuu-para-entender-el-cerebro/>

Shetty, B. (2022, 24 de agosto). What Is the Curse of Dimensionality? *Built In*.
<https://builtin.com/data-science/curse-dimensionality>

Tamburro, A. (2025, 22 de julio). When LLMs Try to Reason: Experiments in Text and Vision-Based Abstraction. *Towards Data Science*.
<https://towardsdatascience.com/when-llms-try-to-reason-experiments-in-text-and-vision-based-abstraction/>

Recursos Web y Documentación

Afshine Amidi. (s.f.). Recuperado de <https://www.mit.edu/~amidi/>

Belcic, I., & Stryker, C. (s.f.). Catastrophic Forgetting. *IBM*. Recuperado el 12 de octubre de 2025, de <https://www.ibm.com/think/topics/catastrophic-forgetting>

Caballar, R., & Stryker, C. (s.f.). Neuromorphic computing. *IBM*. Recuperado el 12 de octubre de 2025, de <https://www.ibm.com/think/topics/neuromorphic-computing>

Convolution Neural Network (Very Basic). (s.f.). *Kaggle*. Recuperado el 12 de octubre de 2025, de <https://www.kaggle.com/discussions/general/171197>

Dengyu-Wu/snncutoff: SNN Cutoff Evaluation. (s.f.). *GitHub*. Recuperado el 12 de octubre de 2025, de <https://github.com/Dengyu-Wu/SNNCutoff>

MLU-Explain. (s.f.). Recuperado el 12 de octubre de 2025, de <https://mlu-explain.github.io/>

Python, R. (s.f.). Machine Learning With Python. Recuperado el 12 de octubre de 2025, de <https://realpython.com/learning-paths/machine-learning-python/>

Yang, X. (s.f.). Convolutional Neural Network: How is it different from the other networks? *Xiaozhou's Notes*. Recuperado el 12 de octubre de 2025, de <https://yangxiaozhou.github.io/data/2020/09/24/intro-to-cnn.html>

Wikipedia

Contributors to Wikimedia projects. (2025, 22 de agosto). Vladimir Keilis-Borok. *Wikipedia*.
https://en.wikipedia.org/wiki/Vladimir_Keilis-Borok

Contributors to Wikimedia projects. (2025, 9 de septiembre). Johnson–Lindenstrauss lemma. *Wikipedia*. https://en.wikipedia.org/wiki/Johnson%E2%80%93Lindenstrauss_lemma

Contributors to Wikimedia projects. (2025, 12 de septiembre). Oscillatory neural network. *Wikipedia*. https://en.wikipedia.org/wiki/Oscillatory_neural_network

Contributors to Wikimedia projects. (2025, 30 de septiembre). Neural oscillation. *Wikipedia*. https://en.wikipedia.org/wiki/Neural_oscillation

Wikimedia, C. de los proyectos. (2023, 2 de mayo). Rectificador (redes neuronales). *Wikipedia*. [https://es.wikipedia.org/wiki/Rectificador_\(redes_neuronales\)](https://es.wikipedia.org/wiki/Rectificador_(redes_neuronales))

Wikimedia, C. de los proyectos. (2025, 6 de mayo). AlphaFold. *Wikipedia*. <https://es.wikipedia.org/wiki/AlphaFold>

Cursos y Material Educativo

Artificial Intelligence > 1. Introduction and Scope. (s.f.). *Class Central*. Recuperado el 12 de octubre de 2025, de <https://www.classcentral.com/classroom/mit-ocw-6-034-artificial-intelligence-fall-2010-40938>

Videos

3Blue1Brown. (s.f.). [Canal de YouTube].

Sautoy, M. du. (2021). The paradox at the heart of mathematics: Gödel's Incompleteness Theorem [Video]. *TED Talks*. https://www.ted.com/talks/marcus_du_sautoy_the_paradox_at_the_heart_of_mathematics_godel_s_incompleteness_theorem?lng=es&subtitle=en

Proyectos y Repositorios

Kaggle. <https://www.kaggle.com/code/nexocodai/metrics>

Colab  **My_LLM.ipynb**

10. ANEXOS

La falta de aprendizaje real en los LLMs

Autor: Bruno Visdómine Minguito

Curso 2025/2026 — IES Arquitecto Ventura Rodríguez

ANEXO A: EJEMPLOS TÉCNICOS Y CASOS DE ESTUDIO

A.1 Ejemplo de Deducción

Un ejemplo simple del funcionamiento de la deducción es: visto A deducimos B. Un ejemplo con palabras sería: Todas las alubias de esta bolsa son blancas; Esas alubias pertenecen a esta bolsa; Entonces esas alubias son blancas

A.2 Ejemplo de Inducción

Desde el punto de la lógica esto se expresa de la siguiente manera: Observamos A, Observamos B, inducimos que $A > B$. Con palabras: Estas alubias pertenecen a esta bolsa; Estas alubias son blancas; Entonces todas las alubias son blancas.

A.3 Funcionamiento Técnico de Modelos de Reconocimiento de Imágenes

Modelos como aquellos que se utilizan para reconocer imágenes "aprenden" al introducir las imágenes del objeto que se desea detectar y relacionarlo con una etiqueta como caballo. Al final se intenta predecir el objeto de la imagen introduciéndola en el modelo para observar el output. Este output o probabilidades numéricas para un objeto como caballo puede reforzarse si el modelo se equivoca o acierta para aumentar la precisión mediante técnicas como el algoritmo de retropropagación.

A.4 Ejemplo Técnico de Sobreajuste

Un buen ejemplo sería conectar todos los puntos de una gráfica pasando por cada uno en vez de dibujar una línea recta que nos muestre la relación entre los puntos.

A.5 Caso de Estudio: Predicción de Terremotos con Big Data

Esto ocurrió por ejemplo en un proyecto que pretendía predecir terremotos con datos históricos sobre seísmos e información geográfica detallada sobre energías de tensión y demás fenómenos que ocurren bajo la superficie, fracasaron penosamente a pesar de encajar perfectamente todos los datos anteriores. Esto ocurre porque según comentan varios geólogos de renombre como David Bowman discípulo de Vladimir Keilis-Borok (geofísico matemático y sismólogo ruso), no hay una teoría que explique que sucede a lo largo de las líneas de falla y por lo tanto no se puede predecir con exactitud cuándo van a friccionar y generar movimiento.

A.6 Formulación Lógica de la Abducción

Abducción: $A > B$, observamos B, deducimos que A es verdadera. Esto es una falacia desde las reglas de la lógica.

Con ejemplos: Todas las alubias de esta bolsa son blancas; Estas alubias son blancas; Entonces estas alubias pertenecen a esta bolsa. Hay que tener en cuenta que esto es un ejemplo muy simple.

ANEXO B: DETALLES TÉCNICOS DE TRANSFORMERS

B.1 Proceso Técnico de Generación

1º Input de los tokens 2º Se pasan por una serie de capas y operaciones con matrices como en el perceptrón multicapa 3º Se van obteniendo probabilidades de tokens que se van uniando y generando constantemente.

B.2 Representación Vectorial de Tokens

Cada token se asocia a unos números que están en una determinada dimensión, o lo que es lo mismo hay el mismo número de números para cada palabra, por ejemplo podemos decir que "ca" = [3.0, 6.3, 2.4, 4.5, 6.2] por lo que esta palabra se representa con un vector de dimensiones 5 x 1. Este token se encuentra en un espacio de 5D, donde también se encuentran todo el resto de tokens. Las relaciones en este espacio acaban modificándose cuando el tokenizador se entrena para acabar pareciéndose lo más fielmente a la distribución estadística en el corpus de entrenamiento. Por ejemplo si usamos palabras, las palabras edificio y rascacielos se encontrarán cerca en ese espacio 5D y los números de sus vectores serán similares, lo mismo con rey y reina etc.

Ejemplo de proximidad semántica:

"edificio" $\approx$ [2.1, 5.8, 3.2, 4.1, 5.9] "rascacielos" $\approx$ [2.3, 5.7, 3.4, 4.0, 6.1]

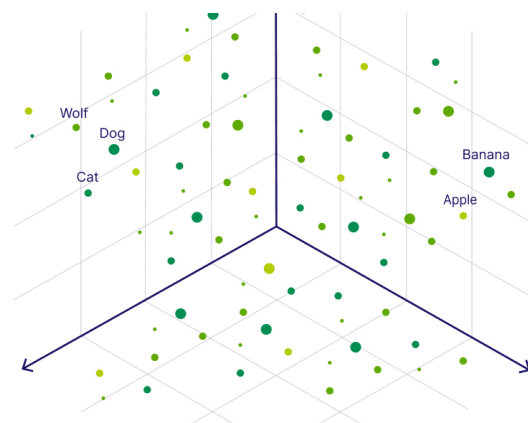
Tokens

6

Characters

21

print a "hello world"



B.3 Teorema de Johnson-Lindenstrauss y Dimensionalidad

Formalización matemática: Para un conjunto de n puntos en $\mathbb{R}^d$, existe una proyección en $\mathbb{R}^k$ donde $k = O(\log n / \epsilon^2)$ que preserva distancias dentro de un factor $(1 \pm \epsilon)$.

En palabras más simples, en pocas d hay infinidad de tokens ya que si se ponen en 90° el número de dimensiones limita el número de tokens o vocabulario sin embargo si se ponen en un rango de $89 - 91^\circ$ en altas dimensiones el vocabulario crece exponencialmente según el teorema de Johnson-Lindenstrauss.

B.4 Conexiones Residuales y Normalización

Tras cada subcapa se emplea una conexión residual que consiste en sumar el input original al output de la primera de las subcapas y a la segunda tras los cálculos, además se normaliza el resultado para que las computaciones no sean extremas y los gradientes (x

numero que se suma o resta al valor actual del parámetro) que haya que usar para ajustar los "pesos" o parámetros no sean enormes. En el caso de los primeros modelos todos los outputs tenían 512 dimensiones, este es el output por subcapa:

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

Esta fórmula representa:

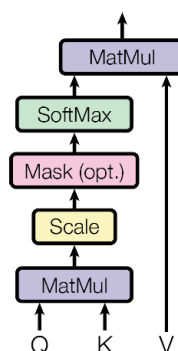
- **Input** (o x): el vector de entrada a la subcapa
- **Sublayer(Input)**: la transformación aplicada por la subcapa (ya sea atención o feed-forward)
- **Input + Sublayer(Input)**: la conexión residual (suma del input original con el output de la subcapa)
- **LayerNorm(...)**: la normalización de capa aplicada al resultado de la suma

B.5 Funcionamiento Detallado del Decodificador

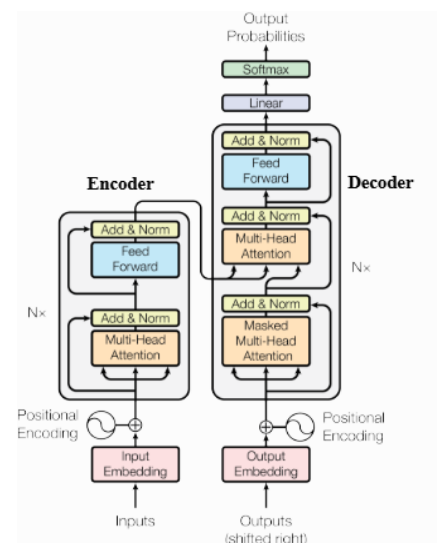
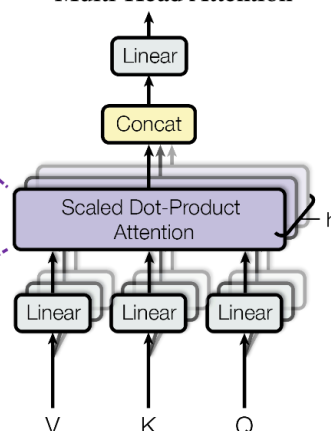
Columna derecha: el Decodificador Esta parte genera la salida paso a paso, usando la información del encoder. Cada bloque de decodificador tiene:

- 1- Masked Multi-Head Self-Attention: evita mirar tokens futuros (autorregresivo).
- 2- Encoder-Decoder Attention: se conecta con las salidas del encoder (usa como contexto).
- 3- Capa feed-forward, residual y normalization igual que el encoder

Scaled Dot-Product Attention



Multi-Head Attention



Resumen decodificador: En el decodificador del Transformer, la información fluye a través de tres subcapas principales dentro de cada bloque. Por ejemplo, si el modelo está traduciendo "I am" al francés y ya ha generado "Je", ese "Je" entra como entrada parcial. Primero pasa por la subcapa de masked multi-head self-attention, que impide que el modelo mire palabras futuras como "suis" mientras está generando. Luego, el resultado pasa por la subcapa de encoder-decoder attention, donde se combina con la información producida por el codificador (que recibió como entrada "I am"). Aquí es donde el modelo puede alinear "Je" con "I" y usar esa conexión para decidir cómo seguir. Finalmente, la información combinada se pasa por una red feed-forward para refinar la representación.

Ejemplo de flujo en traducción:

Input: "I am" →

Encoder → Representación contextual

Decoder input: "Je" → Masked Self-Attention

→ Encoder-Decoder Attention (alinea "Je" con "I")

→ FFN → Predicción: "suis"

Cada una de estas subcapas tiene además conexiones residuales y normalización. Todo esto se repite en cada uno de los bloques del decoder (usualmente seis), permitiendo al modelo generar traducciones o secuencias palabra por palabra de manera coherente.

B.6 Fórmula del Mecanismo de Atención

$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$; donde:

Q (Query): matriz de consultas, dimensión $(n \times d_k)$

K (Key): matriz de claves, dimensión $(m \times d_k)$

V (Value): matriz de valores, dimensión $(m \times d_v)$

d_k : dimensión de las claves

n: longitud de la secuencia de consulta

m: longitud de la secuencia de clave/valor Interpretación:

1. QKT: producto matricial que calcula similitudes entre todas las queries y keys
2. División por $\sqrt{d_k}$: escalado para evitar valores extremos en softmax
3. softmax: normalización para obtener pesos de atención (suman 1)
4. Multiplicación por V: suma ponderada de los valores

B.7 Descripción Técnica de Q, K y V

En el mecanismo de atención, cada token de la secuencia se convierte en tres vectores distintos:

- 1- Q (Query) – lo que quiere saber un token.
- 2- K (Key) – lo que ofrece cada token (como una descripción).
- 3- V (Value) – la información final que se usará si se selecciona ese token.

B.8 Proceso Paso a Paso del Mecanismo de Atención

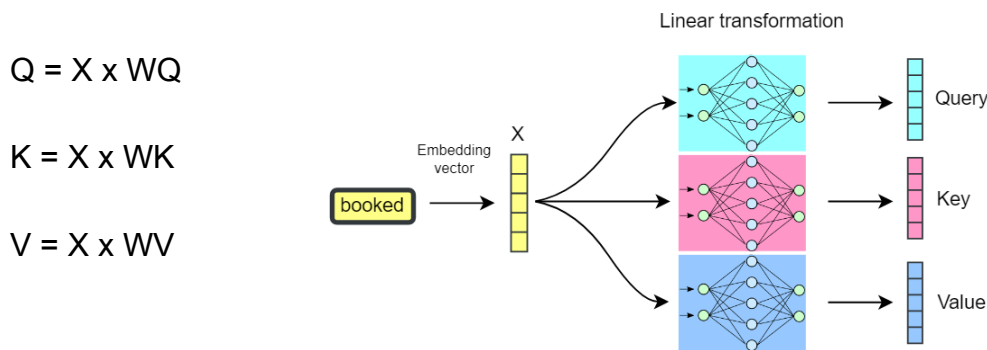
El mecanismo de atención funciona así:

- 1- Se comparan todos los queries (Q) con todos los keys (K) para calcular cuánto debe prestar atención cada token a los demás. Esto se hace con un producto escalar $Q \times K^T$, que da una matriz de atención.
- 2- Se aplica softmax para normalizar los valores → así se obtiene el peso que cada token debe tener. (En vez de tener números gigantes como en la siguiente imagen)
- 3- Finalmente, esos pesos se usan para hacer una suma ponderada de los values (V) → eso da el nuevo vector de salida que se transforma en un token.

B.9 Cálculo de Matrices Q, K y V

Q, K y V se calculan con los vectores de los tokens y tres matrices diferentes que son previamente entrenadas para correctamente transformar los tokens en los vectores que se relacionan estadísticamente, estas proyecciones con matrices son las que ayudan a

encontrar nuevas relaciones entre los tokens o palabras aparte de su cercanía en el embedding space. Aquí X es la matriz con todos los vectores que representan a cada token. Las W son las matrices entrenables.



B.10 Ejemplo Práctico: "Yo no como manzanas"

Por ejemplo en la frase "Yo no como manzanas", si nos centramos en cómo funciona el mecanismo de atención con "manzanas" veremos que:

Primero se calcula el vector query de manzanas en este caso solo con su vector y no con X entero.

Segundo se calculan todos los keys del resto de tokens

Tercero se multiplica el query por cada key de los otros tokens de la manera: $Q(\text{manzanas}) \times K(\text{Yo})$, $Q(\text{manzanas}) \times K(\text{no})$, $Q(\text{manzanas}) \times K(\text{como})$ para saber la relación estadística que mantiene manzana con cada palabra y qué palabra está más o menos relacionada con manzana en este caso.

Cuarto se divide entre la raíz cuadrada de las dimensiones de las keys d_K para que el modelo no crezca de manera desproporcionada, y además se aplica la función softmax al resultado para minimizar los pesos para que no exploten o lleguen a mínimos.

Finalmente se multiplica esta matriz que contiene las relaciones estadísticas creadas entre los tokens o palabras que se han introducido por la matriz de los valores que son una representación alternativa de los vectores originales de cada token.

La matriz WV aprende cómo convertir el embedding base en una versión útil para ser "leída" por otros tokens, o potenciar un cierto aspecto de la representación del token. La matriz W $\square$ aprende a potenciar aspectos específicos de cada token. En un espacio de 128 dimensiones: Dimensión 100: relacionada con negación/afirmación Dimensión 47: relacionada con temporalidad Dimensión 83: relacionada con agentividad. Si nos encontramos con el token "no" (ya que es una palabra bastante corta) la matriz WV puede potenciar la dimensión 100 o muchas otras dependiendo de la necesidad en la frase.

B.11 Fórmula de la Red Feedforward

Se llevan a cabo dos transformaciones con dos matrices diferentes y se añaden dos sesgos como en las típicas redes neuronales.

x: vector de un token

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

B.12 Aclaración: codificación posicional

Un ejemplo es utilizando la siguiente función que te da un vector de las mismas dimensiones que las del modelo lo que permite sumarlos directamente y representa la posición del token:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d(modelo)}}\right) \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d(modelo)}}\right)$$

Supongamos que queremos codificar la posición 3 en una frase con una codificación de 4 dimensiones. Aplicamos funciones seno y coseno con diferentes frecuencias según cada dimensión:

Dim 0: $\sin(3) \approx 0.1411$, **Dim 1:** $\cos(0.1687) \approx 0.9858$, **Dim 2:** $\sin(0.03) \approx 0.0299$, **Dim 3:** $\cos(0.0053) \approx 0.99999$

Resultado del vector de posición:

$$PE(3) \approx [0.1411, 0.9858, 0.0299, 0.99999]$$



Y este vector se lo sumamos al vector original que representa al token como por ejemplo el siguiente $[0.3, 0.7, -0.1, 0.5]$ nos queda $[0.4411, 1.6858, -0.0701, 1.49999]$.

B.13 Aclaración: Atención vs Auto-atención

Conviene hacer una aclaración: Atención: relaciona dos secuencias diferentes (traducción). Auto-atención: relaciona elementos de una misma secuencia entre sí (generación). Ambos usan el mismo mecanismo matemático, pero con diferentes fuentes de información.

Mecanismo	¿De dónde vienen Q, K y V?	¿Qué permite?
Atención "normal"	Q de un lado, K y V de otro	Relacionar dos secuencias distintas (Traducción)
Auto-atención	Q, K y V del mismo lugar (misma secuencia)	Relacionar tokens dentro de la misma secuencia (Generación de texto)

B.14 Ejemplo Completo del Proceso de Generación

(Cuando se comparan los vectores se utiliza el dot product pero a gran escala con una matriz que contiene todos los vectores de los tokens y el vector de salida, si el resultado se aproxima a 0 para un token en concreto es que no hay relación si se acerca a 1 es que están muy relacionados).

En este modelo como hemos visto anteriormente se usaron vectores ya aprendidos para los tokens, que los representan en un espacio de 512 dimensiones, se utilizan capas con pesos aprendidos para convertir los tokens en vectores al principio, y otra capa para predecir los siguientes tokens a partir de los vectores de salida (a veces compartiendo pesos con el embedding original). En un ejemplo:

1. Entrada (embedding inicial) "gato" entra al modelo y se convierte en un vector fijo usando la matriz de embeddings. Este vector inicial representa "gato" de forma general, sin contexto.

2. Pasa por el transformer (transformación contextual) Ese vector se transforma a lo largo de las capas del modelo. Ahora, la representación final de "gato" ya incluye el contexto de la frase (por ejemplo, si viene después de "El" y antes de "duerme", esa información está codificada). Así que ahora tienes un vector contextualizado: h_t

3. Comparación con todos los tokens del vocabulario Ese vector h_t se compara con todos los vectores del vocabulario (uno por cada token). Esto se hace con un producto escalar que mide cuánta "afinidad" hay entre el contexto actual y cada posible token.

4. Softmax → probabilidades Se aplica softmax a todos esos valores para obtener probabilidades. El token con mayor probabilidad es el que el modelo predice como siguiente. Por ejemplo:

$$P(\text{perro}) = 9.97/297.29 = 0.034 \text{ (3.4\%)}$$

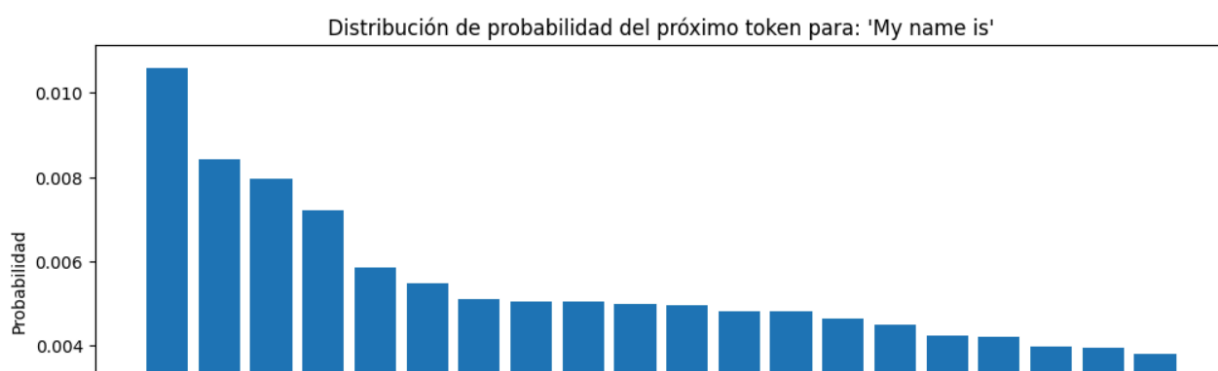
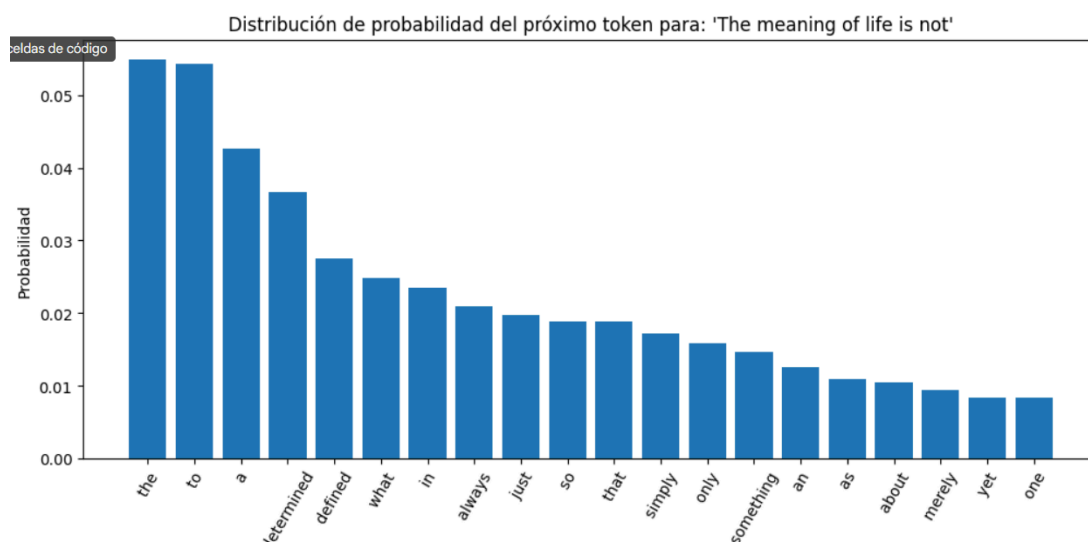
$$P(\text{gato}) = 164.02/297.29 = 0.552 \text{ (55.2\%)}$$

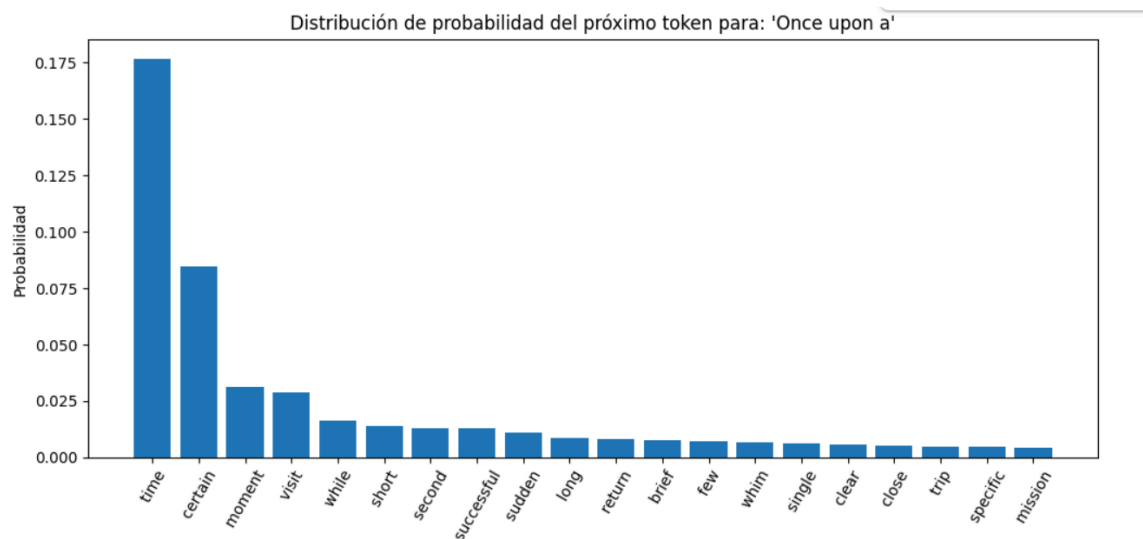
$$P(\text{come}) = 1.49/297.29 = 0.005 \text{ (0.5\%)}$$

$$P(\text{duerme}) = 121.51/297.29 = 0.409 \text{ (40.9\%)}$$

$$P(.) = 0.30/297.29 = 0.001 \text{ (0.1\%)}$$

Unnormalized Attention Pattern						Normalized Attention Pattern					
+3.53	+0.80	+1.96	+4.48	+3.74	-1.95	1.00	0.75	0.69	0.92	0.46	0.00
-∞	-0.30	-0.21	+0.82	+0.29	+2.91	0.00	0.25	0.08	0.02	0.01	0.46
-∞	-∞	+0.89	+0.67	+2.99	-0.41	0.00	0.00	0.24	0.02	0.22	0.02
-∞	-∞	-∞	+1.31	+1.73	-1.48	0.00	0.00	0.00	0.04	0.06	0.01
-∞	-∞	-∞	-∞	+3.07	+2.94	0.00	0.00	0.00	0.00	0.24	0.48
-∞	-∞	-∞	-∞	-∞	+0.31	0.00	0.00	0.00	0.00	0.00	0.00





B.15 Funcionamiento del proceso de multi-atención

En lugar de una única función de atención, se ejecutan h funciones en paralelo:

$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)$

Donde $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

Parámetros del modelo original:

$h = 8$ cabezas

$d_k = d_v = d_{\text{model}} / h = 512 / 8 = 64$

Dimensión de salida: $d_{\text{model}} = 512$

Ventajas de Múltiples Cabezas Cada cabeza puede aprender diferentes tipos de relaciones:

Cabeza 1: Relaciones sintácticas (sujeto-verbo) "El gato" → "duerme" (peso alto)

Cabeza 2: Correferencias pronominales "María" → "ella" (peso alto)

Cabeza 3: Dependencias semánticas "comprar" → "tienda" (peso alto)

Cabeza 4: Relaciones temporales "ayer" → verbos en pasado (peso alto)

B.16 Resultados propios

En las siguientes imágenes entrenadas en diferentes conjuntos de datos que estaban en diferentes idiomas ilustran cómo los transformers intentan imitar la estructura de las frases, ya sea en inglés, francés o español, pero si se estudian las imágenes con más detenimiento se descubre la falta completa de sentido en las respuestas, producto de la falta de tiempo y recursos. En cualquier caso se puede apreciar como los LLMs no comprenden sino que escupen los patrones de los datos con los que han sido entrenados.

```
→ loading 54/54 tensors
Resumed from epoch 2
```

Prompt: Dans les années

Generated:
Dans les années autorités collectif

```
Loading checkpoint: checkpoints/ckpt_epoch_1_best.pt
→ loading 86/86 tensors
Resumed from epoch 1
```

Prompt: User: What is an hour?

Generated text:
User: What is an hour?
1: Yeah technology is the racing of infilt sectrate people do? Na is first-b is a conven There are talk to heard f
rom the together. They having from friends, Fefine to make up decisions that you would be sour create a brered to
tell needs, mannerable or insurance. Willing one them up for our former censity, stable idea of your numer, seriou
s were weession with sh

```
Loading checkpoint: checkpoints/ckpt_epoch_3.pt
🔥 Cargando 70/70 tensores compatibles.
Resumed from epoch (3, None)
```

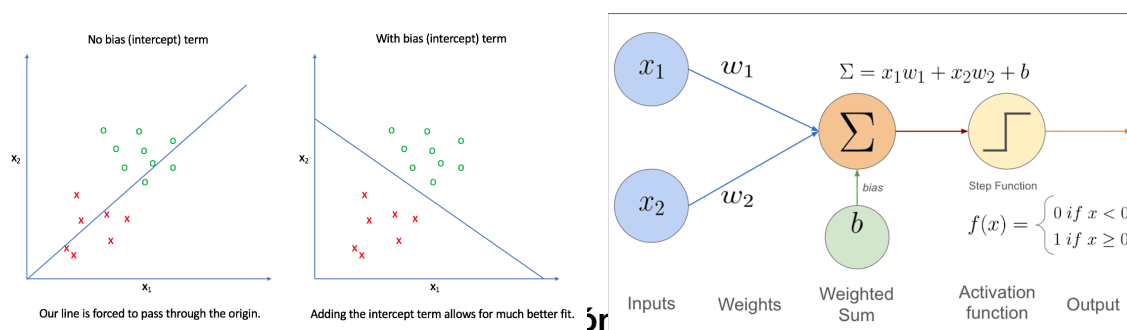
Prompt: User: ¿Que pasa si mi gato es grande?

Generated text:
User: ¿Que pasa si mi gato es grande?
Bot: Sí, pero solo si desaparecela bajo corres muy y la reind de calcetines viejoeven para cantarse, todo para la m
ateria excadilillas sí misma brilla no la filoso y pequeñas f

ANEXO C: DETALLES TÉCNICOS DEL PERCEPTRÓN

C.1 Estructura Técnica del Perceptrón

La estructura era la siguiente, tenías dos inputs que se multiplicaban por los pesos, se sumaban sus resultados y se añadía un sesgo que desplazaba la función de activación, finalmente se pasaba el resultado por una función de activación y si el resultado era mayor o igual que 0 o un umbral cualquiera había activación (1), si no, no había activación (0).



Un ejemplo de aplicación sería un dataset con altura y peso de 5000 personas clasificadas en obeso/no obeso. Tras el entrenamiento, el modelo "aprende" a trazar una frontera lineal de separación y puede predecir a qué clase pertenece un nuevo individuo (por ejemplo, -1= no obeso, 1= obeso).

C.3 Funcionamiento Técnico del Entrenamiento y Predicción

El entrenamiento consiste en ajustar los pesos y el sesgo. El perceptrón calcula un producto escalar entre el vector de pesos y el de entradas, y según su signo determina la clase. Geométricamente, esto equivale a que el perceptrón va ajustando la orientación del hiperplano hasta que consigue dividir correctamente los dos conjuntos de datos. El sesgo desplaza ese hiperplano, afinando la separación. Los pesos suelen inicializarse con valores aleatorios pequeños y se corrigen iterativamente con la regla de aprendizaje.

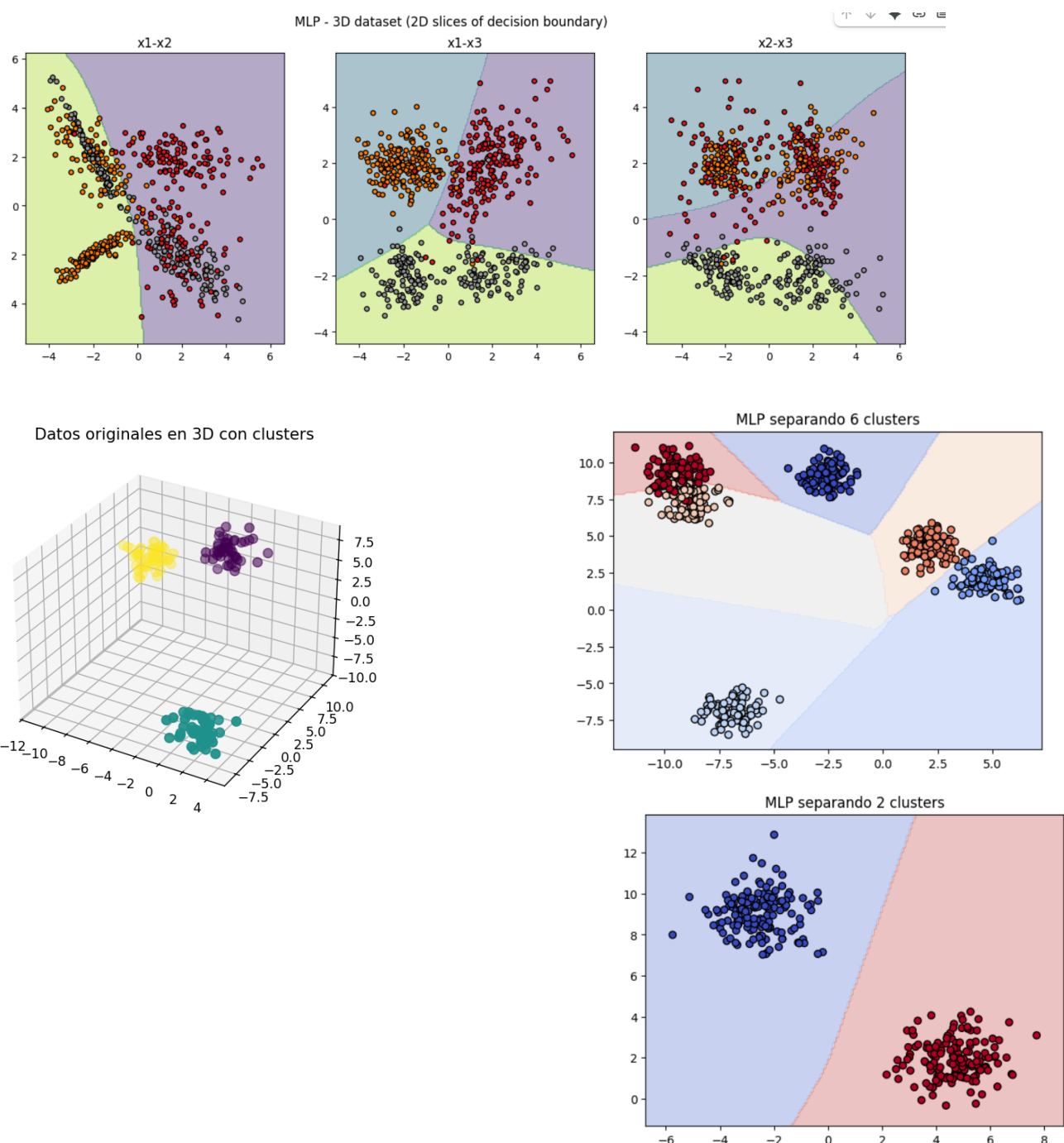
Una vez entrenado, introducir un vector de entrada (como altura y peso) implica multiplicarlo por los pesos, sumar el sesgo y aplicar la función de activación. En el perceptrón clásico esta función es un escalón: convierte el resultado en -1 o 1 dependiendo de qué lado del

hiperplano caiga el punto. En más dimensiones, el perceptrón aprende un hiperplano de separación, generalizando a conjuntos de datos con muchas características.

ANEXO D: DETALLES TÉCNICOS DEL PERCEPTRÓN MULTICAPA

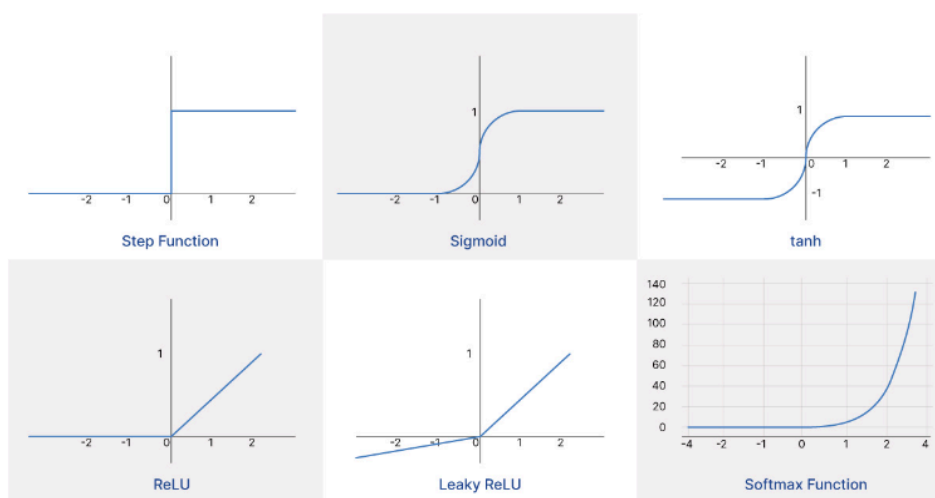
D.1 Ejemplo de Aplicación del Perceptrón Multicapa

El perceptrón multicapa permitió la creación de hiperplanos en mayores dimensiones y por tanto la separación de más clases de datos.



D.2 Función Sigmoide y Funciones de Activación

Función Sigmoide: Cuando t en este ejemplo tiende a menos infinito S tiende a 0, pero cuando t tiende a infinito S tiende a 1. Lo que se hace es multiplicar todas las conexiones con cada neurona anterior y sumar todo, ese valor se le asigna a la próxima neurona. Existen otros tipos de funciones de activación.



D.3 Cálculo de Pesos y Sesgos Totales

Si calculamos, con las dos capas intermedias teniendo 16 neuronas, en total habría 13.002 pesos y sesgos. Aquí aprender quiere decir que la red neuronal aprenda los valores de los pesos y sesgos que darán lugar a predicciones estadísticas acertadas. Los valores de activación, sesgos y pesos se pueden agrupar en vectores y matrices para que el cálculo resulte más sencillo computacionalmente.

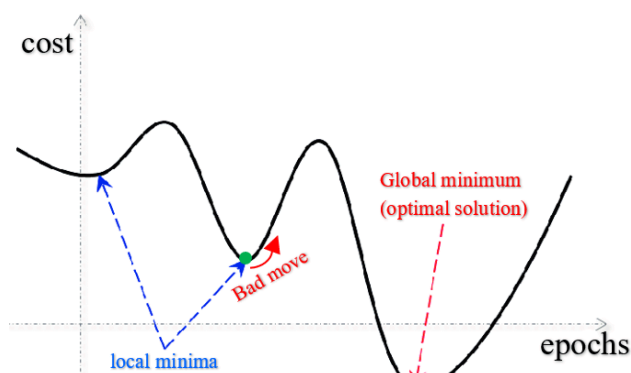
D.4 Función de Coste (Mean Squared Error)

Por ejemplo en la siguiente (Mean squared error) se resta el valor predicho al valor real y se eleva al cuadrado, se hace con cada una de las predicciones y se suman hasta llegar al número N y se hace la media.

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Annotations for the equation:

- $\frac{1}{N}$: average over all results
- $\sum_{i=1}^N$: sum over all results
- y_i : true y
- $\hat{y}_i$: estimate of y
- $(y_i - \hat{y}_i)^2$: makes result quadratic



D.5 Desarrollo Matemático Completo de la Retropropagación

Los parámetros $C(w_1, b_1, w_2, b_2, w_3, b_3)$ representan, por ejemplo, los pesos y sesgos que hay que ajustar durante el entrenamiento. Cada neurona tiene un valor de activación, denotado como $a^{(L)}$ para la última capa, $a^{(L-1)}$ para la capa anterior, $a^{(L-2)}$ para la siguiente, y así sucesivamente.

Consideremos un ejemplo concreto: si la última neurona tiene un valor $a^{(L)} = 0.8$ pero nosotros queremos que $a^{(L)} = 1.0$, entonces se calcula el coste como:

$$\text{Coste} = (0.8 - 1.0)^2$$

Sabemos que el valor de activación de la última capa viene dado por:

$$a^{(L)} = \sigma[w^{(L)} \cdot a^{(L-1)} + b^{(L)}]$$

donde σ representa la función de activación. Para simplificar la notación, lo que se encuentra dentro de la función de activación se puede expresar como $z^{(L)}$:

$$z^{(L)} = w^{(L)} \cdot a^{(L-1)} + b^{(L)}$$

Todo esto es suponiendo que las conexiones entre neuronas sean lineales, con una sola neurona por capa y una sola conexión, para simplificar el desarrollo.

Cálculo de Gradientes mediante la Regla de la Cadena

Lo que hacemos es calcular el impacto de la variación de cada parámetro en el coste final. Esto se escribe como $\partial \text{Coste} / \partial w^{(L)}$, que sería la relación entre el coste final y el peso final. Realmente se calcula de la siguiente manera:

$$\partial \text{Coste} / \partial w^{(L)} = (\partial \text{Coste} / \partial a^{(L)}) \cdot (\partial a^{(L)} / \partial z^{(L)}) \cdot (\partial z^{(L)} / \partial w^{(L)})$$

Esto se llama la **regla de la cadena**, ya que el peso y el coste no están directamente relacionados, por lo que se calculan las derivadas de los parámetros que se encuentran entre ellos hasta conectar ambos.

El resultado es:

$$\partial \text{Coste} / \partial \mathbf{w}^{(L)} = 2[\mathbf{a}^{(L)} - \mathbf{y}] \cdot \sigma'[\mathbf{z}^{(L)}] \cdot \mathbf{a}^{(L-1)}$$

donde $\sigma'[\mathbf{z}^{(L)}]$ es la derivada de la función de activación evaluada en $\mathbf{z}^{(L)}$.

Promedio sobre Múltiples Ejemplos

Como este proceso se repite con varios ejemplos de datos durante el entrenamiento, se suman todos los gradientes y se hace la media de la siguiente manera:

$$\partial \text{Coste} / \partial \mathbf{w}^{(L)} = (1/n) \sum_{k=0}^{n-1} \partial \text{Coste}^{(k)} / \partial \mathbf{w}^{(L)}$$

Esta es la media calculada con todos los ejemplos de entrenamiento.

Gradiente respecto al Sesgo

El mismo proceso se hace con el sesgo $\mathbf{b}^{(L)}$. Tras derivar, obtenemos una ecuación más sencilla, ya que nos quedamos con:

$$\partial \text{Coste} / \partial \mathbf{b}^{(L)} = 2[\mathbf{a}^{(L)} - \mathbf{y}] \cdot \sigma'[\mathbf{z}^{(L)}] \cdot \mathbf{1}$$

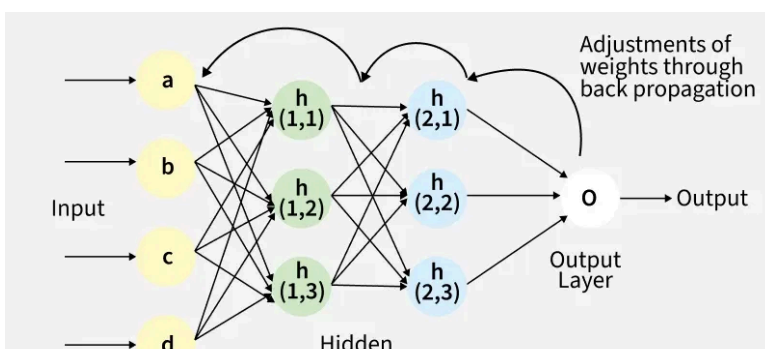
El factor 1 aparece porque el término $\mathbf{b}^{(L)}$ en $\mathbf{z}^{(L)}$ sólo está sumando ($\partial \mathbf{z}^{(L)} / \partial \mathbf{b}^{(L)} = \mathbf{1}$).

Propagación hacia Capas Anteriores

Para propagar el gradiente hacia las capas anteriores, calculamos:

$$\partial \text{Coste} / \partial \mathbf{a}^{(L-1)} = 2[\mathbf{a}^{(L)} - \mathbf{y}] \cdot \sigma'[\mathbf{z}^{(L)}] \cdot \mathbf{w}^{(L)}$$

Este cálculo nos da el ratio entre el coste y el valor de activación de la neurona anterior. Si iteramos con la regla de la cadena hacia atrás, podemos llegar a los primeros pesos y sesgos y ajustar la red entera mediante este proceso de retropropagación.



Cost function:

$$C_0 = \sum_{j=0}^{n_L-1} (a_j^{(L)} - q_j)^2$$

Activation in layer L :

$$a_j^{(L)} = \sigma(z_j^{(L)})$$

Backpropagation chain rule:

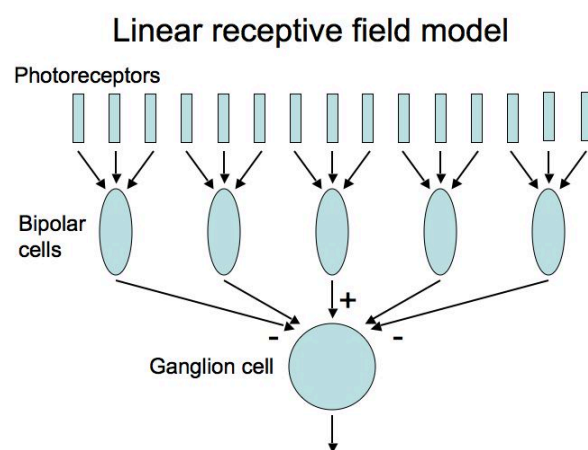
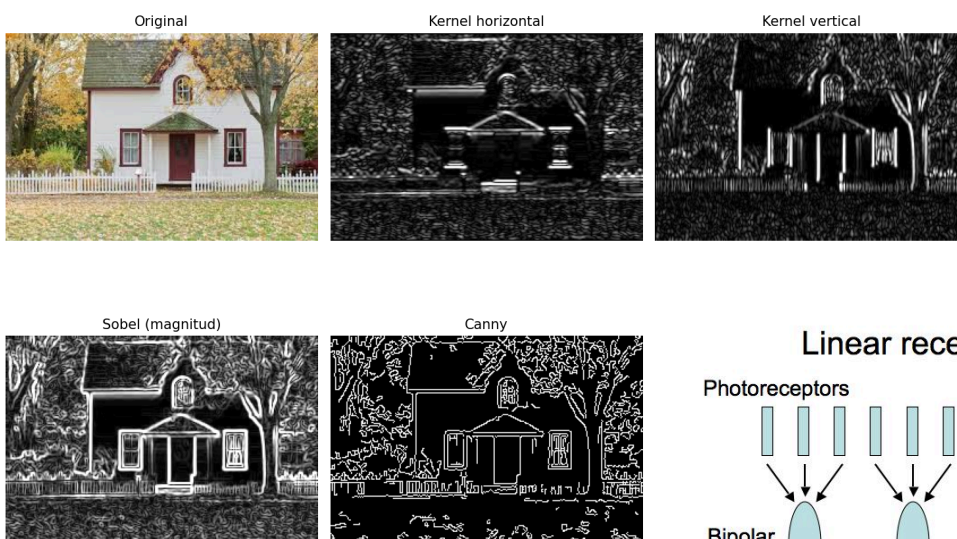
Generalización a Redes Más Complejas

Todo esto ocurre, como he mencionado antes, en un caso simplificado. Si añadimos más neuronas en las capas intermedias, los cálculos se complican un poco, pero la idea fundamental es la misma. Se utiliza una notación extra (típicamente subíndices adicionales) para señalar a cada neurona y su posición específica dentro de cada capa, pero el principio de la regla de la cadena se aplica de manera análoga.

ANEXO E: DETALLES TÉCNICOS DE CNN, RNN Y MODELOS DE DIFUSIÓN

E.1 Funcionamiento Técnico Detallado de las CNN

El funcionamiento se organiza en etapas sucesivas: capas de convolución aplican kernels que generan mapas de características detectando patrones específicos, capas de activación (típicamente ReLU) introducen no linealidad, capas de pooling reducen resolución manteniendo información esencial, y al apilar estas operaciones se construyen jerarquías de representaciones abstractas que finalmente se introducen en capas totalmente conectadas para clasificación.

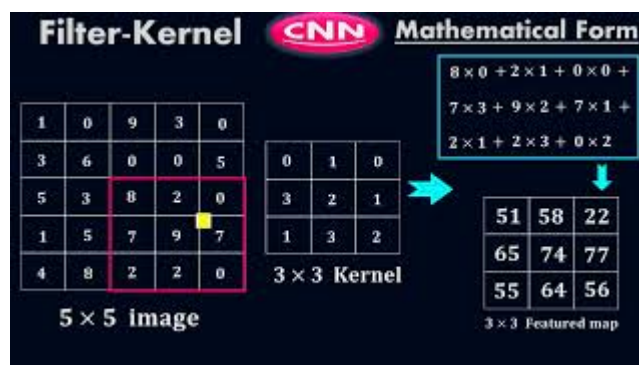


E.2 Proceso de Entrenamiento Supervisado de CNN

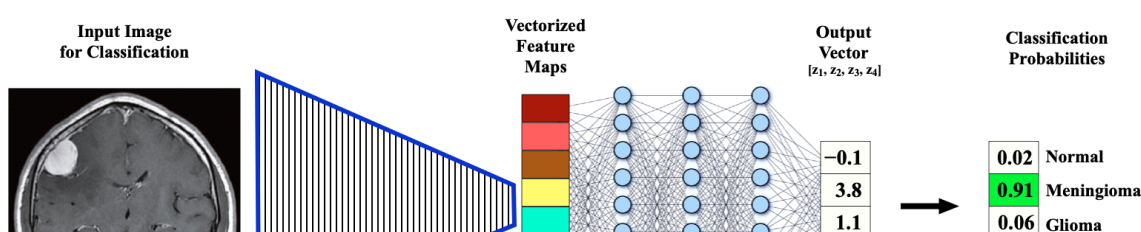
El entrenamiento de una CNN sigue el paradigma del aprendizaje profundo supervisado. El proceso comienza con un conjunto de datos de imágenes etiquetadas. La red, con pesos inicializados al azar, realiza predicciones que se comparan con las etiquetas reales mediante una función de pérdida (por ejemplo, entropía cruzada en clasificación). A continuación, se aplica el algoritmo de retropropagación del error (backpropagation) junto con descenso por gradiente estocástico (SGD) o variantes como Adam para ajustar los pesos de los filtros y las conexiones. Con cada iteración, los filtros aprenden a detectar patrones útiles para resolver la tarea. Tras muchas épocas de entrenamiento, la red alcanza la capacidad de generalizar y reconocer objetos en nuevas imágenes no vistas.

$$\text{Horizontal Kernal Filter} = \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$$

$$\text{Vertical Kernal Filter} = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}$$



Precisión en test: 97.64%



E.3 Funcionamiento Técnico Completo de los Modelos de Difusión

Los modelos de difusión son modelos generativos que crean imágenes de alta calidad a partir de ruido aleatorio mediante un proceso iterativo que combina CNNs y Transformers. Funcionan en dos fases: el proceso forward añade ruido gaussiano progresivamente a la imagen original hasta destruirla, y el proceso reverse utiliza una red neuronal para revertir este ruido paso a paso. La arquitectura típica emplea una U-Net basada en CNNs para capturar detalles locales y características espaciales, mientras que integra mecanismos de cross-attention tipo Transformer para incorporar contexto global y condicionamiento textual, donde la imagen ruidosa actúa como query y los embeddings del texto como keys y values. Durante el entrenamiento con pares imagen-texto, el modelo aprende a predecir el ruido en cada paso minimizando la diferencia entre el ruido real y el predicho, ajustando así la distribución condicional. En la generación, parte de ruido aleatorio y aplica iterativamente la red de denoising guiada por el prompt, produciendo imágenes coherentes que reflejan las relaciones aprendidas entre lenguaje y características visuales.

ANEXO F: Neuroplasticidad vs. IA - Comparación Cuantitativa

F.1 Aprendizaje de Nueva Tarea

Cerebro Humano

Cuando el cerebro humano se enfrenta a una nueva tarea, como reconocer un objeto desconocido (por ejemplo, una herramienta nunca vista), solo necesita entre 3 y 5 exposiciones para comenzar a consolidar ese conocimiento. El tiempo de consolidación completo toma aproximadamente 24 a 48 horas, proceso en el cual el sueño juega un papel fundamental. El mecanismo biológico que permite este aprendizaje incluye la potenciación a largo plazo (LTP), la neurogénesis en el hipocampo y el fortalecimiento selectivo de sinapsis específicas.

Una vez consolidado, este conocimiento puede retenerse durante décadas con solo refuerzos ocasionales. Lo más notable es que la interferencia con el conocimiento previo es mínima, es decir, aprender algo nuevo raramente causa que olvidemos información ya establecida.

LLM con Fine-tuning

En contraste, cuando un modelo de lenguaje grande (LLM) debe aprender a reconocer un nuevo concepto en texto, requiere entre 1,000 y 10,000 ejemplos para ajustarse adecuadamente. El tiempo de entrenamiento oscila entre 2 y 8 horas utilizando GPUs de alto rendimiento. El mecanismo empleado se basa en la retropropagación y la actualización global de pesos, proceso que carece de la selectividad sináptica característica del cerebro biológico.

La retención del nuevo conocimiento dura únicamente hasta el próximo proceso de fine-tuning, momento en el cual puede perderse o degradarse significativamente. Además, la interferencia con el conocimiento previo es alta, registrándose pérdidas de información entre el 40% y el 80% del conocimiento original, fenómeno conocido como olvido catastrófico.

F.2 Formación de Nuevas Conexiones

Sinaptogénesis (Cerebro)

El cerebro humano contiene aproximadamente 16×10^9 neuronas en la corteza cerebral, cada una de las cuales establece alrededor de 7,000 sinapsis con otras neuronas, lo que resulta en un total aproximado de 10^{14} conexiones sinápticas. Durante periodos de aprendizaje activo, el cerebro forma entre 10^8 y 10^9 nuevas sinapsis cada día. Este proceso es continuo y altamente selectivo, fortaleciendo únicamente aquellas conexiones que resultan relevantes para las experiencias y aprendizajes en curso.

Simultáneamente, ocurre un proceso de poda sináptica que elimina aproximadamente 10^7 conexiones por día, correspondientes a sinapsis que no se utilizan o que han perdido su funcionalidad. Este balance entre formación y eliminación de sinapsis mantiene al sistema nervioso en un equilibrio dinámico, optimizando constantemente su estructura para reflejar las necesidades actuales del organismo.

Red Neuronal Artificial Estática

GPT-3, uno de los modelos de lenguaje más conocidos, contiene 175×10^9 parámetros. Sin embargo, una vez completado su entrenamiento inicial, el número de nuevos parámetros que se forman es exactamente cero. El modelo permanece estático en su arquitectura fundamental. Cuando se emplean técnicas como LoRA (Adaptación de Bajo Rango) para añadir capacidades específicas, se agregan aproximadamente 10^8 parámetros nuevos por tarea, lo que representa apenas el 0.06% del total de parámetros del modelo.

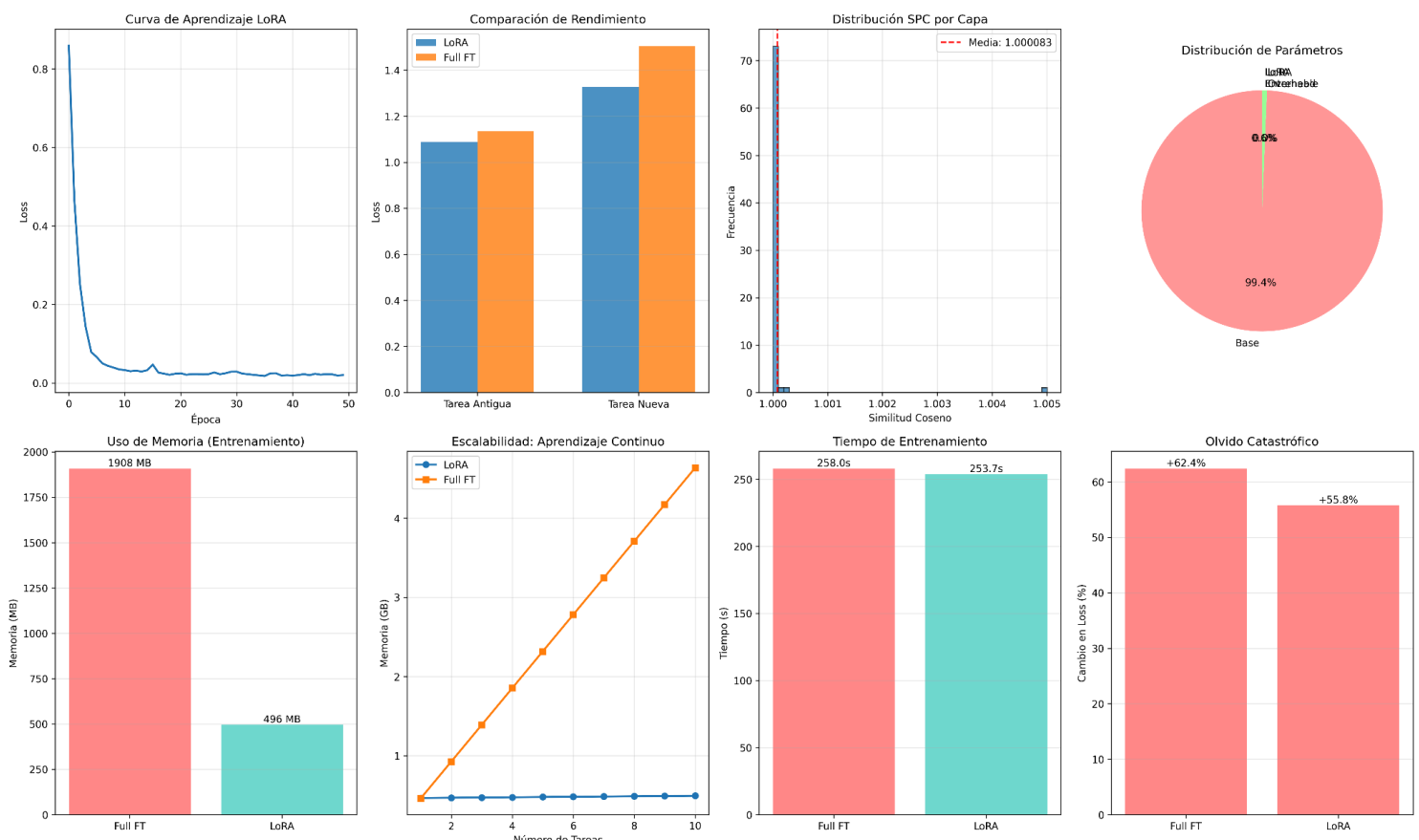
Es importante destacar que estos parámetros adicionales funcionan como módulos externos, no como una integración verdadera en la estructura del modelo. A medida que se añaden más tareas, estos módulos se acumulan de forma lineal sin ningún proceso de consolidación que los integre coherentemente en el conocimiento previo del modelo.

Ratio de plasticidad comparativo

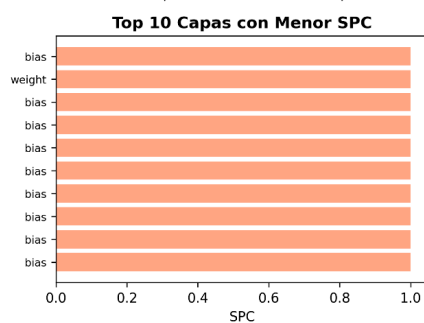
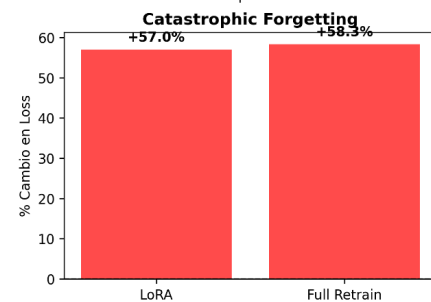
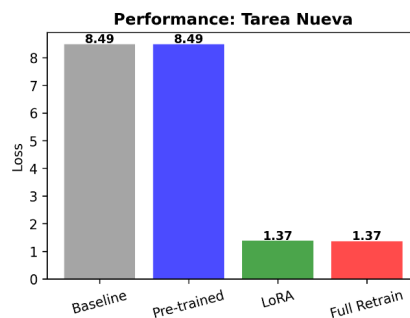
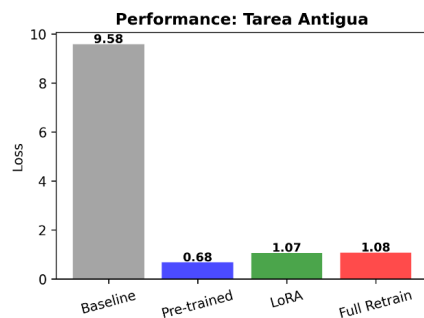
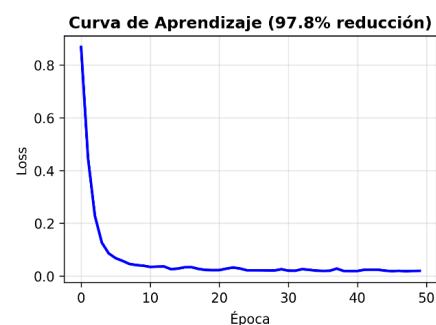
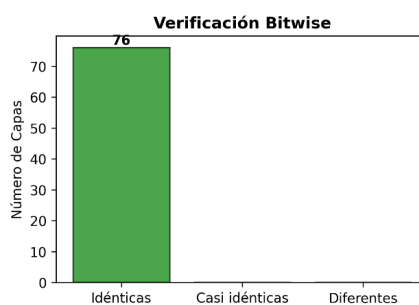
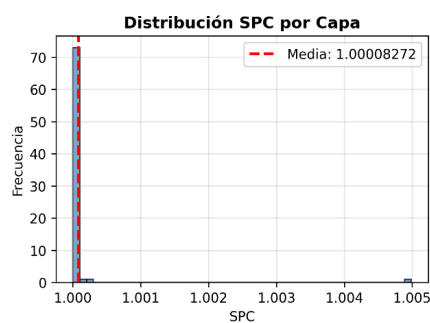
Si calculamos el ratio de plasticidad en el cerebro humano, encontramos que diariamente se modifica aproximadamente el 0.0001% de las sinapsis totales ($10^8 / 10^{14}$). Aunque este porcentaje parece minúsculo, estas modificaciones se consolidan de forma permanente, integrándose completamente en la arquitectura existente. En contraste, un LLM con LoRA añade el 0.06% de parámetros por cada nueva tarea ($10^8 / 175 \times 10^9$), una cifra porcentualmente mayor, pero estos parámetros no se consolidan realmente. En lugar de integrarse, se acumulan externamente como módulos independientes, sin la capacidad de reorganizar o reforzar selectivamente la estructura subyacente del modelo.

Esta comparación revela una diferencia fundamental: mientras el cerebro humano modifica menos en términos porcentuales pero de forma integrada y consolidada, los sistemas artificiales actuales añaden más parámetros pero sin verdadera plasticidad estructural.

ANEXO G: Gráficas y datos



EXPERIMENTO A: LoRA como Memoria Externa vs Plasticidad Real



RESUMEN DEL EXPERIMENTO A: LoRA como Memoria Externa

RESULTADOS CLAVE:

- SPC Global: 1.00008272
- Cambio angular (1-SPC): -0.008272%
- Capas idénticas: 100.0%
- Max diferencia: 0.00e+00

APRENDIZAJE FUNCIONAL:

- Reducción de loss: 97.75%
- Loss inicial: 0.8683
- Loss final: 0.0195

CATASTROPHIC FORGETTING:

- LoRA: +57.00%
- Full Retrain: +58.35%
- Mitigación: 1.35% mejor con LoRA

CONCLUSIÓN:

- ✓ LoRA NO modifica los parámetros base (SPC ≈ 1.0)
- ✓ LoRA SÍ reduce significativamente el loss (~98%)
- ✓ LoRA actúa como MEMORIA EXTERNA, no como plasticidad
- ✓ LoRA mitiga mejor el catastrophic forgetting